

Systemes de gestion de fichiers

Maher SELLAMI

Sommaire

- Introduction
- Organisation logique
- Organisation physique
- Structuration du disque dur
- Réalisation des fonctions d'accès élémentaires
- Sécurité et Protection

Fonctions d'un SGF

- Fichier
 - un objet qui contient un ensemble d'information pour **conservation** et **utilisation** dans un système informatique
 - possède un nom qui permet de le désigner
 - Fonctions d'accès principales: lire, écrire, éventuellement exécuter
- SGF
 - Conservation des fichiers en mémoire secondaire
 - Implantation des fonctions d'accès aux fichiers (ouvrir, fermer, lire, écrire, ...)
 - Assure sécurité et protection: intégrité des informations et respect des droits d'accès
 - Gère l'accès aux supports physiques

Organisation d'un SGF

Un SGF réalise la correspondance entre l'organisation logique et l'organisation physique

Organisation logique

- Vue par l'utilisateur
- Déterminé par condition de
 - Commodité
 - généralité

SGF

Organisation physique

- Liée aux supports physiques
- Déterminé par considération de
 - Économie de places
 - Efficacité d'accès

répertoire={ descripteurs}

TABLES D'IMPLANATION Physiques

Nom externe

Nom interne

Un SGF est organisé de façon hiérarchique : toute action au niveau logique et interprété comme un ensemble d'actions au niveau physique

Exemple

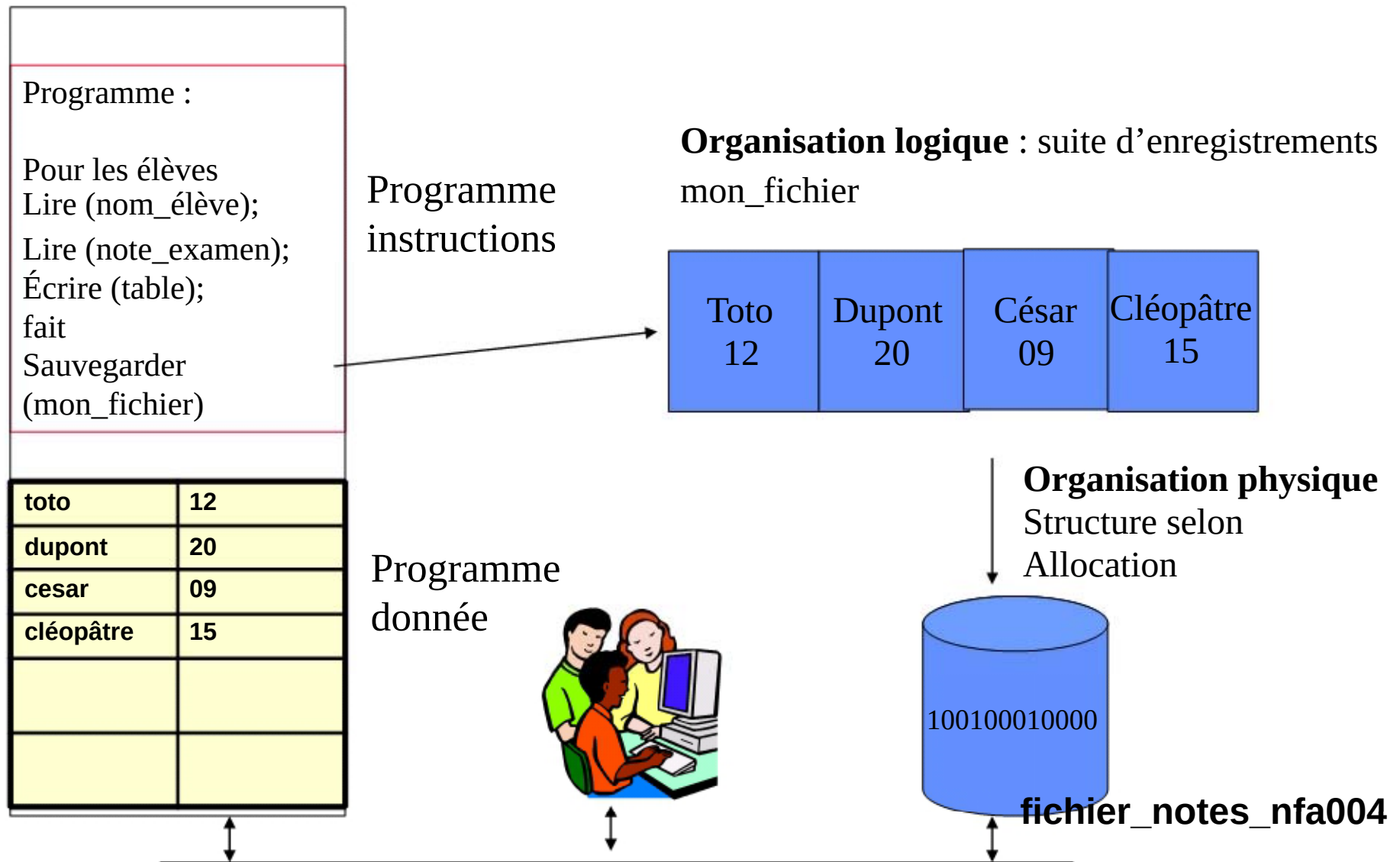


Schéma de l'organisation d'un SGF

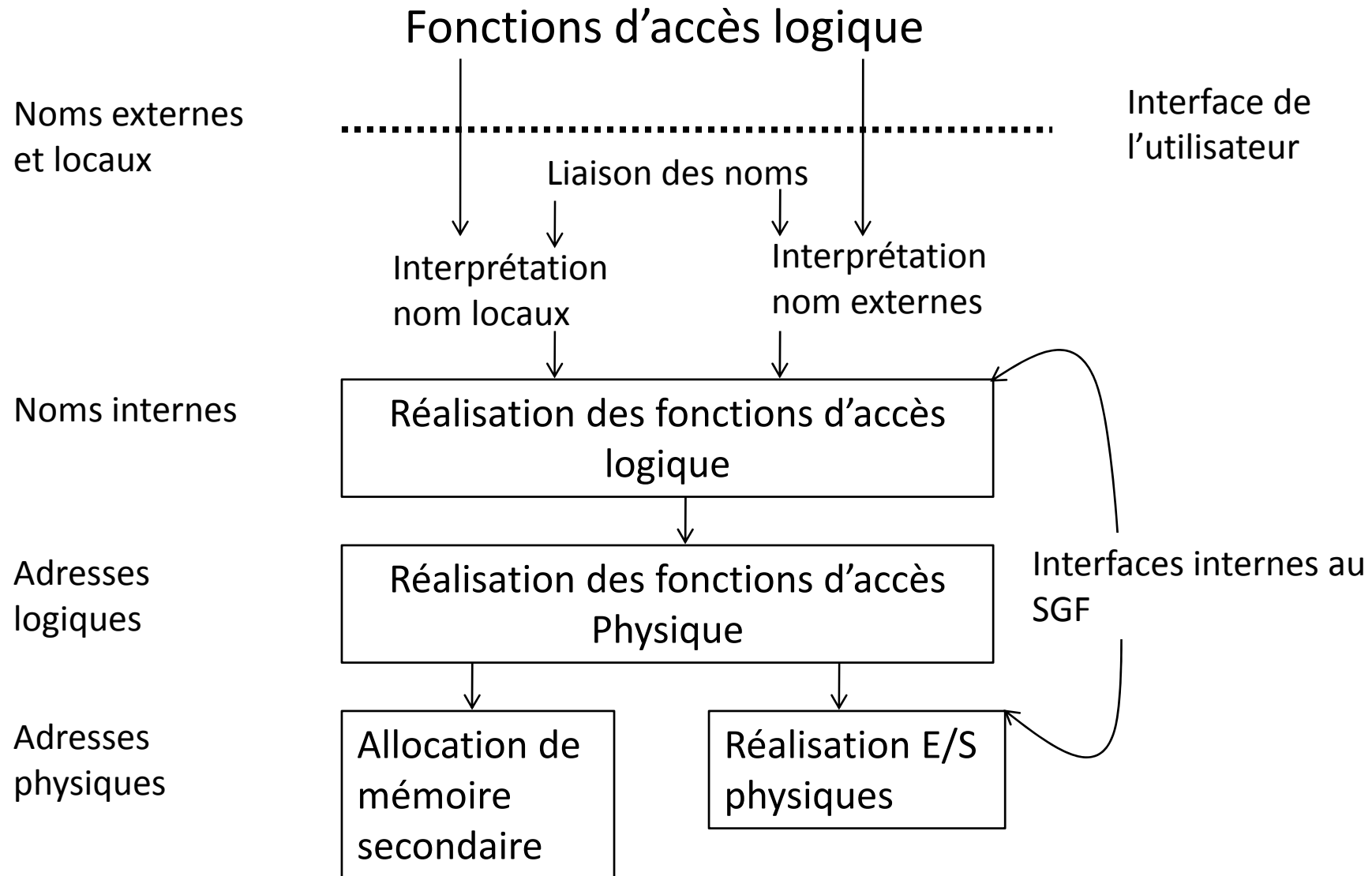
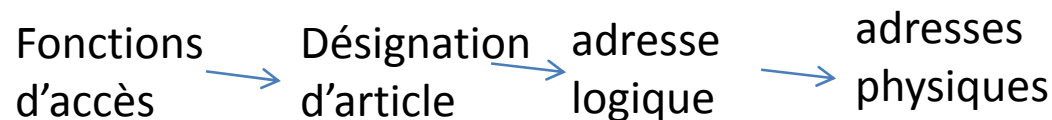


Schéma de l'organisation d'un SGF

- La mise en correspondance entre organisation logique et physique peut comporter une organisation intermédiaire



- Cette étape intermédiaire pour des raisons d'efficacité peut être court-circuitée (unix, windows, ...)

deux cas de figure :

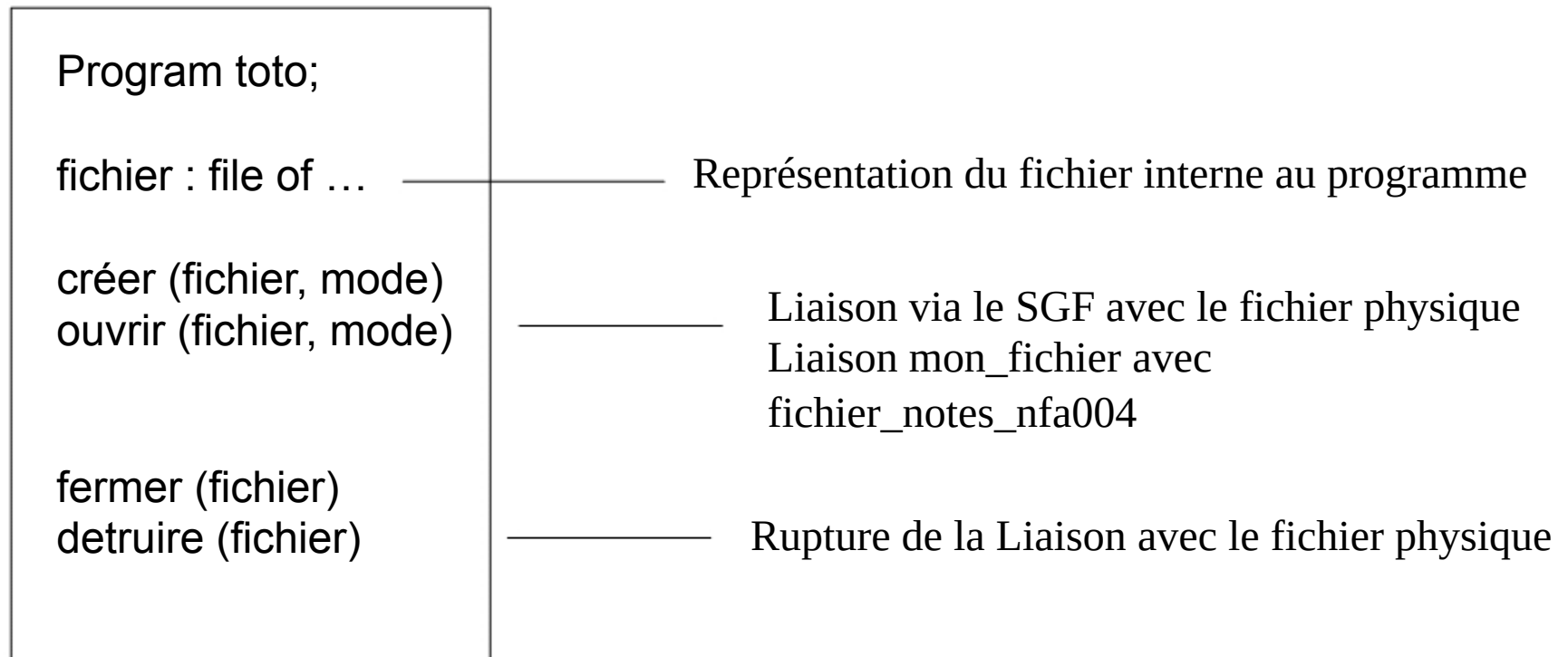
- Le S.E. connaît la structure logique des fichiers (Macintosh, IBM,...)
 - + les possibilités sont plus importantes,
 - le S.E. est plus complexe.
- Le S.E. considère les fichiers comme des flots d'octets (MS-DOS, Unix,)
 - + le système est plus simple,
 - pas d'assurance sur la nature des fichiers

Organisation logique d'un fichier

- vue de l'utilisateur de l'ensemble des données mémorisées sur le support de masse
 - Un type de donnée (programmation)
 - Un ensemble de données groupées sous forme d'enregistrements (articles)
- fichier = { enregistrements }
- Les fonctions d'accès sont spécifiques de l'organisation logique
- A chaque enregistrement correspond une adresse logique
- enregistrement = {champs ou attributs}
- Champs possède un nom et un type
- L'Organisation logique dépend des contraintes auxquelles doivent satisfaire les enregistrements
 - Ordre sur les articles
 - Restriction sur les valeurs des champs
 - Relations entre les attributs des $\neq$ enregistrements

Notion de fichier logique

- En programmation, un fichier logique est un type de donnée sur lequel peuvent être appliquées des opérations spécifiques.



Notion de fichier logique

- Un fichier logique est un ensemble d'enregistrements, désigné par un nom et accessible via des fonctions d'accès.

```
Type élément = record  
    nom-élève : char;  
    note : entier;  
end;
```

enregistrement

Toto 12	Dupont 20	César 09	Cléopâtre 15
------------	--------------	-------------	-----------------

mon_fichier : file of element

Fichier logique :

Attributs du fichier

Nom logique (mon_fichier)

fonctions d'accès

lire (enregistrement)

écrire (enregistrement)

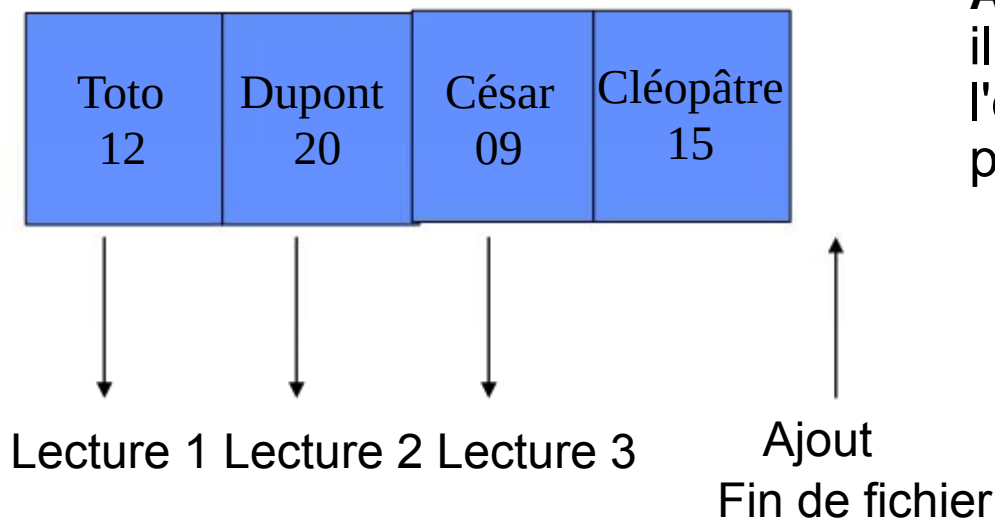
insérer (enregistrement)

supprimer (enregistrement)

Organisation définissant sa structure
logique : **le mode d'accès**

Fichier à mode d'accès séquentiel

- Les enregistrements du fichier ne peuvent être accédés que les uns à la suite des autres.
 - Ouverture du fichier : positionne sur le premier enregistrement
 - Opération de lecture : délivre l'enregistrement courant et se positionne sur le suivant
 - Opération d'ajout : obligatoirement en fin de fichier



Accès à l'enregistrement 3
il faut lire d'abord
l'enregistrement 1,
puis l'enregistrement 2

Exemple : les fichiers dans le langage pascal

```
program acces_fichiers;
```

```
enrg_note = record      nom_eleve : string;  
                        note : integer;  
                        end;
```

Enregistrement

```
t_fichier_notes = file of enrg_note;
```

```
var    mon_fichier : t_fichier_notes;
```

Déclaration fichier logique

```
Begin
```

```
assign(mon_fichier, 'fichier_notes_nfa004'); Mise en correspondance
```

```
reset ( mon_fichier );      (* ouvrir le fichier *)
```

```
while not eof ( nom_fichier ) do
```

```
(* parcourir les éléments du fichier - accès séquentiel*)
```

```
begin
```

```
get ( mon_fichier );
```

```
end;
```

```
(* accès relatif : aller à l'élément 6 *)
```

```
seek (mon_fichier,6) ;
```

```
get (mon_fichier) ;
```

```
(* fermeture automatique en fin de programme *)
```

```
End.
```

Exemple : les fichiers dans le langage C

```
Main() {
```

```
    struct note{ char nom_eleve[30];  
                  float note;  
    } ;
```

Enregistrement

```
    Struct note enrg_note;
```

Déclaration fichier logique

```
    FILE *mon_fichier;
```

Mise en correspondance

```
    mon_fichier=fopen("fichier_notes_nfa004" , "r");
```

```
    while (fread(&enreg_note,sizeof(struct note),1,  
mon_fichier) do
```

```
    (* parcourir les éléments du fichier - accès séquentiel*/
```

```
    {
```

```
    ....
```

```
    }
```

```
    ....
```

```
    /* accès relatif : aller à l'élément 6 */
```

```
    fseek(mon_fichier, sizeof(struct note)*6L,0);
```

```
    ....
```

```
    /* fermeture automatique en fin de programme */
```

```
    }
```

Fichier à mode d'accès direct

- Les fonctions d'accès s'expriment en fonction des attributs
- Les attributs correspondent aux valeurs des différents champs
- Une clé est tout attribut d'un enregistrement dont la valeur peut servir à accéder à l'enregistrement
- 2 organisations possibles:
 - Clé unique
 - Clés multiples

Accès direct: clé unique

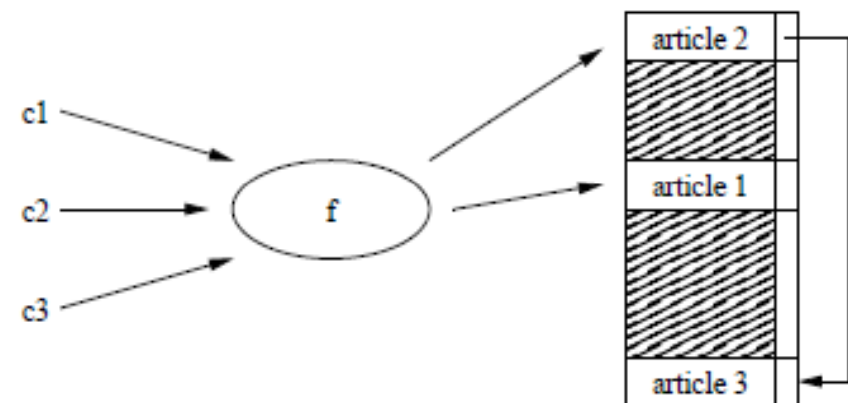
- Chaque article comporte une seule clé qui l'identifie
`cle_trouve=recherche(clé, al)` délivre l'adresse logique ***al*** correspondant à « clé » si `cle_trouve` est vrai
- La fonction recherche sert à réaliser un jeu de fonctions d'accès direct:
 - lire (clé, info)
 - ajouter(clé, info)
 - supprimer(clé, info)
 - modifier(clé, info)
- 2 classes de méthodes :
 - Adressage dispersé (hash code)
 - Fichiers indexés

Accès direct: Adressage dispersé

- On se donne f une fonction de dispersion qui est définie :
 - $0 \leq f(c) \leq \text{nb d'enregistrements}$
 - si $c1 = c2$, alors $f(c1) = f(c2)$,
- On appelle collision, l'existence de deux clés $c1$ et $c2$ telles que $c1 \neq c2$ et $f(c1) = f(c2)$.

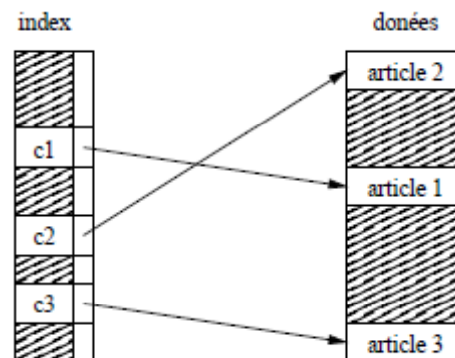
Inconvénients :

- il est difficile de choisir f ,
- sur certains système perte de place,
- réorganisation périodique du fichier.



Adressage direct par indexation

- Ensemble de clés supposé ordonné
- La relation clé-adresse logique est matérialisée par un index



La recherche d'un enregistrement nécessite en moyenne

- $n/2$ lectures si l'index n'est pas trié,
- $\log_2 n$ lectures si l'index est trié.

Adressage direct : clés multiples

- Plusieurs clés pour désigner un article
 - ☞ Une valeur de clé peut correspondre à plusieurs articles
- Une clé primaire est une clé dont la valeur détermine de façon unique un enregistrement
- Technique de base est l'organisation multi-liste
 - ☞ On utilise un index par clé

Adressage direct : clés multiples

Index des références

A135	200
B436	500
R211	200
R322	100
X007	750

Index des auteurs

...	...
JACQUES	500
...	...
PAUL	100
...	...

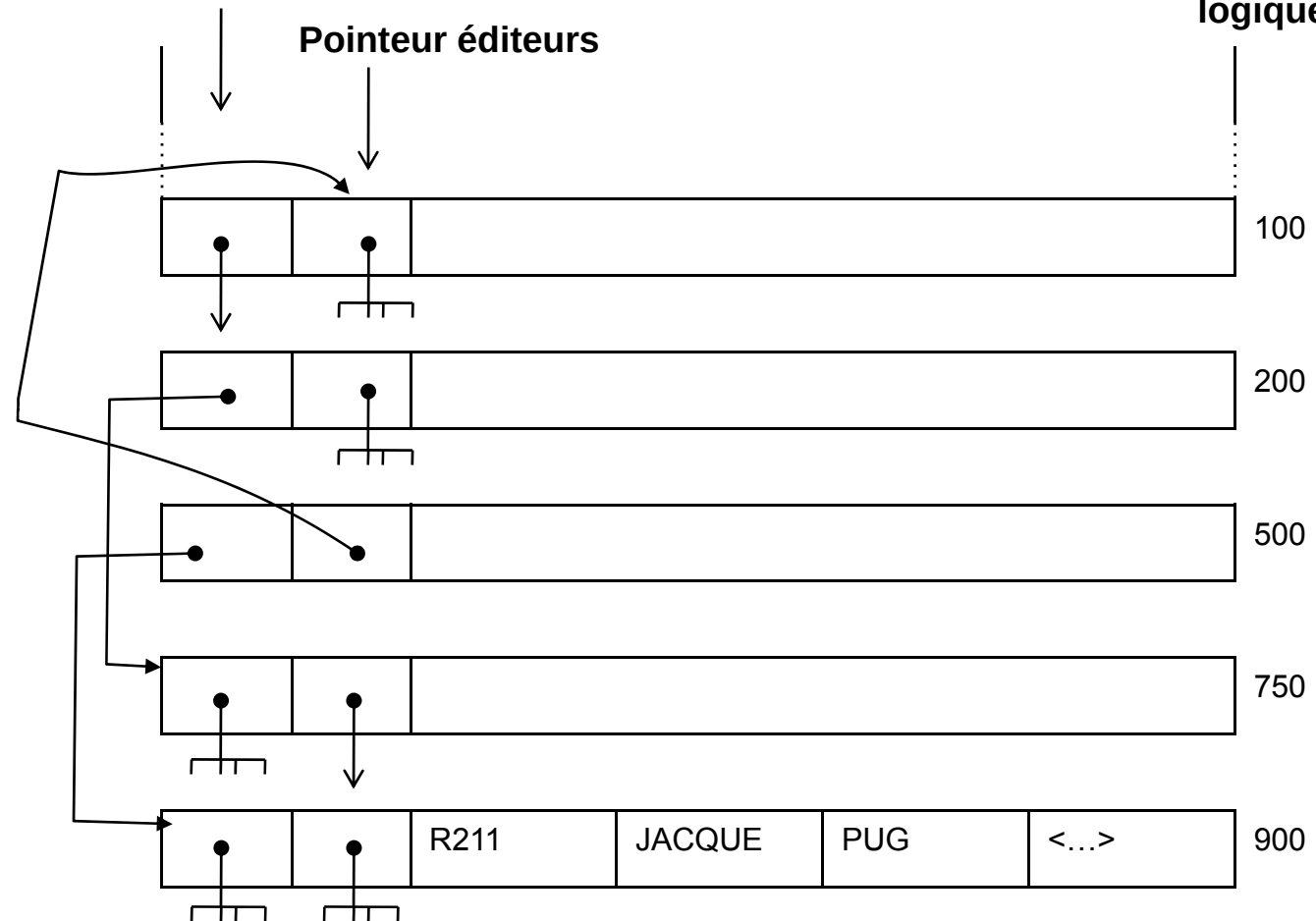
Index des éditeurs

...	...
ARTHAUD	200
...	...
DUNOD	500
...	...
PUG	750

Pointeur auteurs

Pointeur éditeurs

Adresses logiques



Organisation multilistes pour un fichier à clés multiples

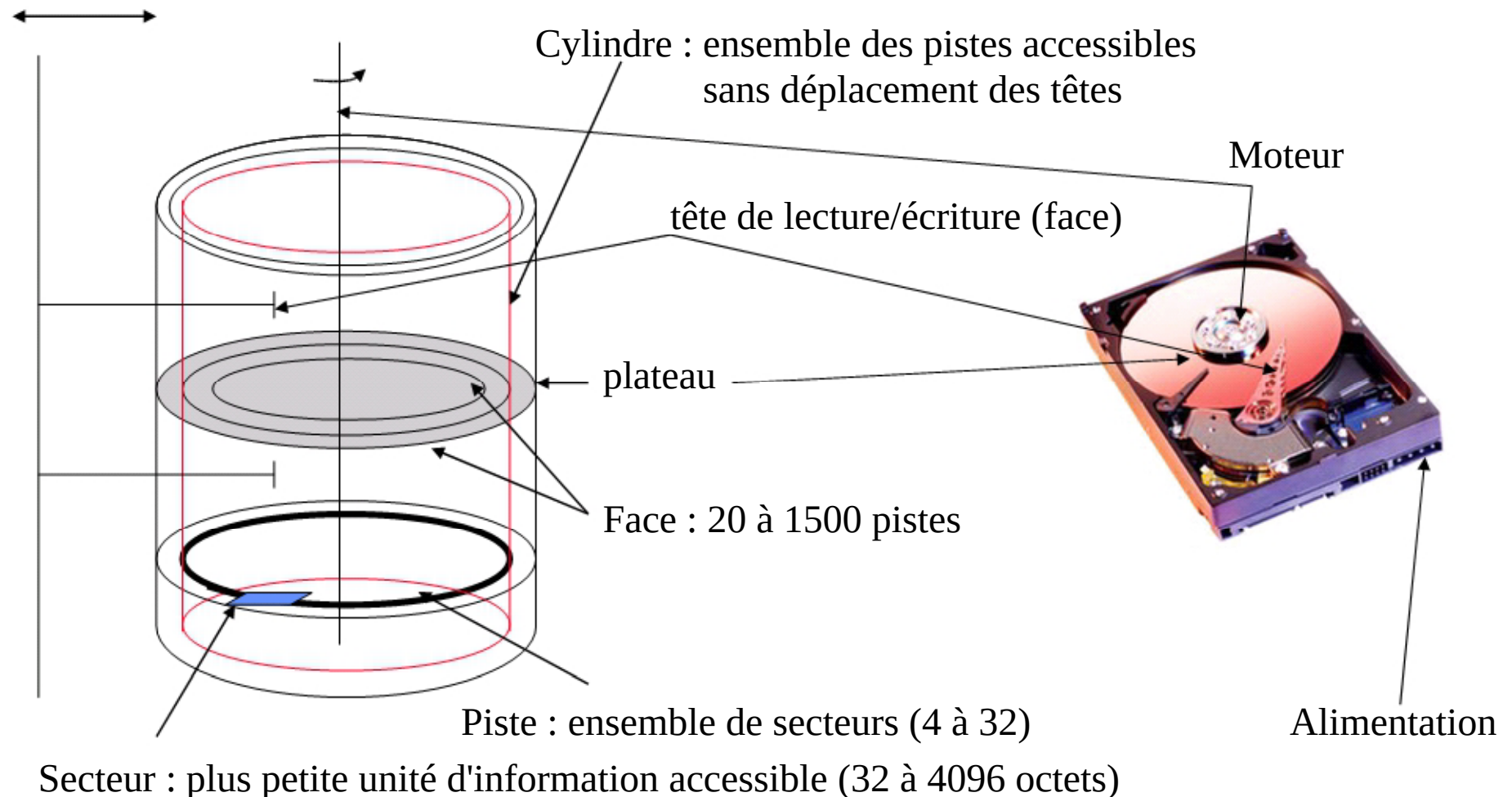
Organisation physique d'un fichier

Problème

- Comment allouer l'espace disque aux fichiers de sorte que:
 - Cet espace disque soit bien utilisé
 - L'accès aux fichiers soit rapide

Structure du disque dur

↳ Adresse physique (secteur) : n°tête, n°cylindre, n°secteur



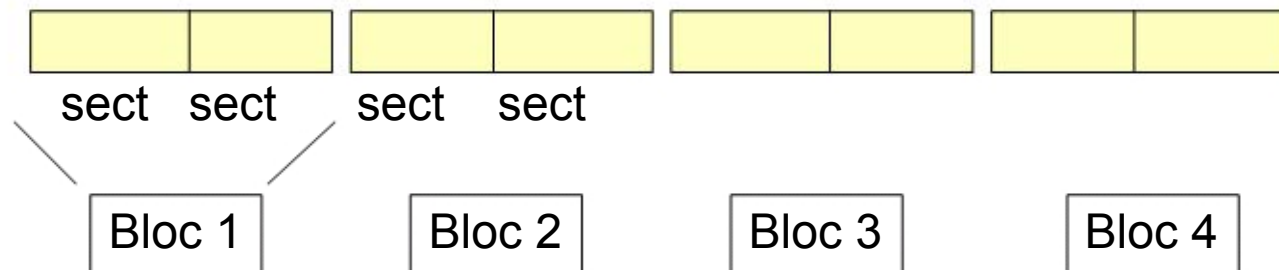
Allocation du disque : le bloc physique

L'unité d'allocation sur le disque dur est le **bloc physique**.
Il est composé de 1 à n secteurs

ex:

1 bloc = 2 secteurs de 512 octets soit 1KO

Les opérations de lecture et d'écriture du SGF se font bloc par bloc



Implantation physique

Programme :

Pour les élèves
Lire (nom_élève);
Lire (note_examen);
Écrire (table);
fait
Sauvegarder
(mon_fichier)

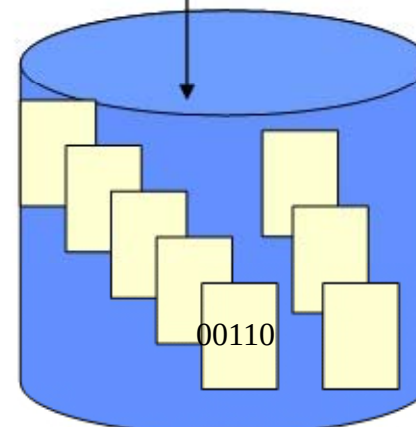
Programme
instructions

Fichier logique : suite d'enregistrements
mon-_fichier

Toto 12	Dupont 20	César 09	Cléopâtre 15
------------	--------------	-------------	-----------------

totot	12
dupont	20
cesar	09
cléopâtre	15

Programme
donnée



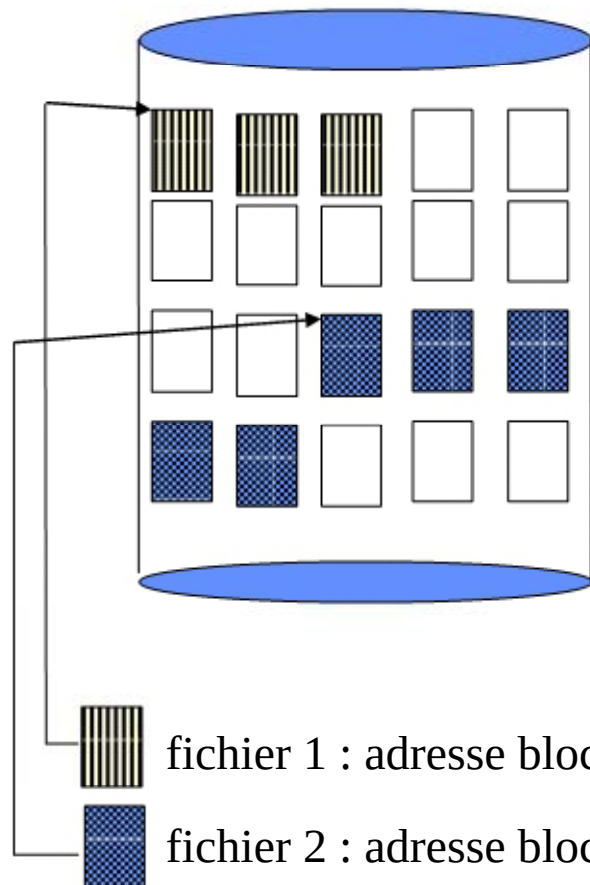
Fichier physique
Ensemble de
blocs physiques
Fichier_notes_nfa
004

Implantation physique

- Un fichier physique est constitué d'un ensemble de blocs physique.
- Il existe plusieurs méthodes d'allocation des blocs physiques :
 - allocation contiguë (séquentielle simple)
 - allocation par zones
 - allocation par blocs chaînés
 - allocation indexée
- ☞ il faut gérer et représenter l'espace libre

Allocation contiguë

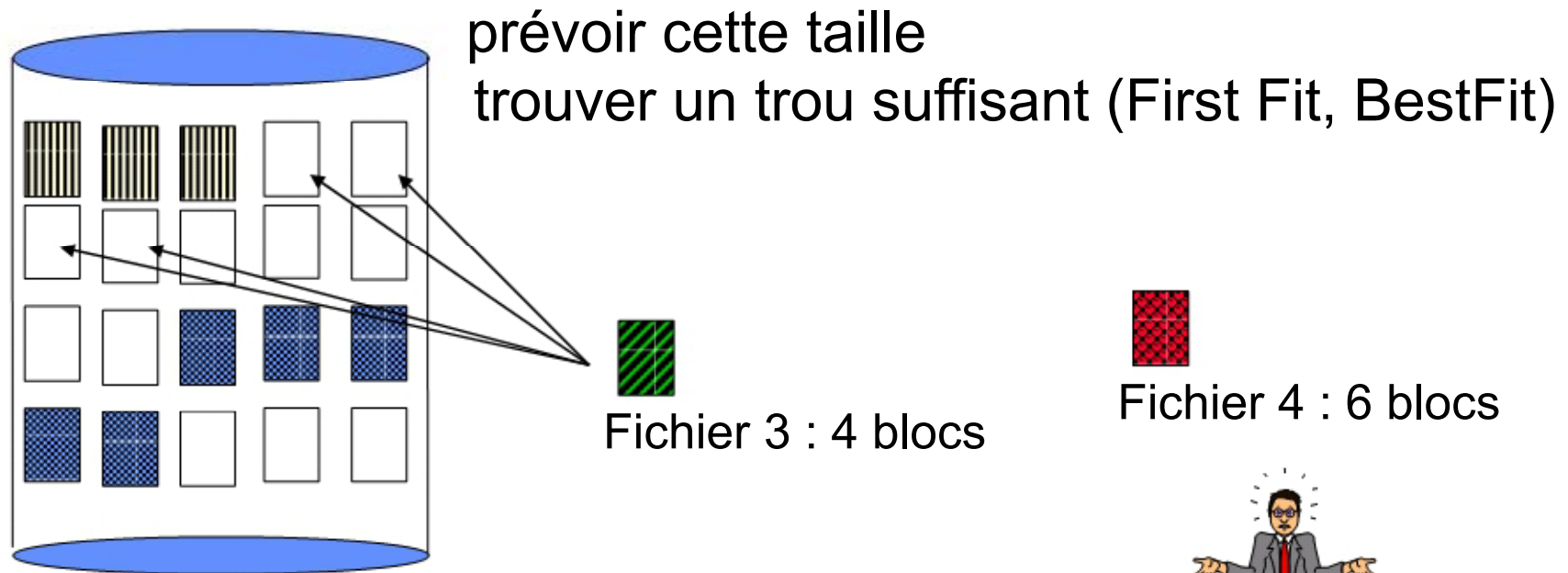
- Un fichier occupe un ensemble de blocs contigus sur le disque



- Bien adapté aux méthodes d'accès séquentielles et directes
- Difficultés :
 - Fragmentation de la mémoire
 - extension du fichier

Allocation contiguë

- création d'un nouveau fichier : il faut allouer un nombre de blocs suffisants dépendant de la taille du fichier



fichier 1 : adresse bloc 1, longueur 3 blocs

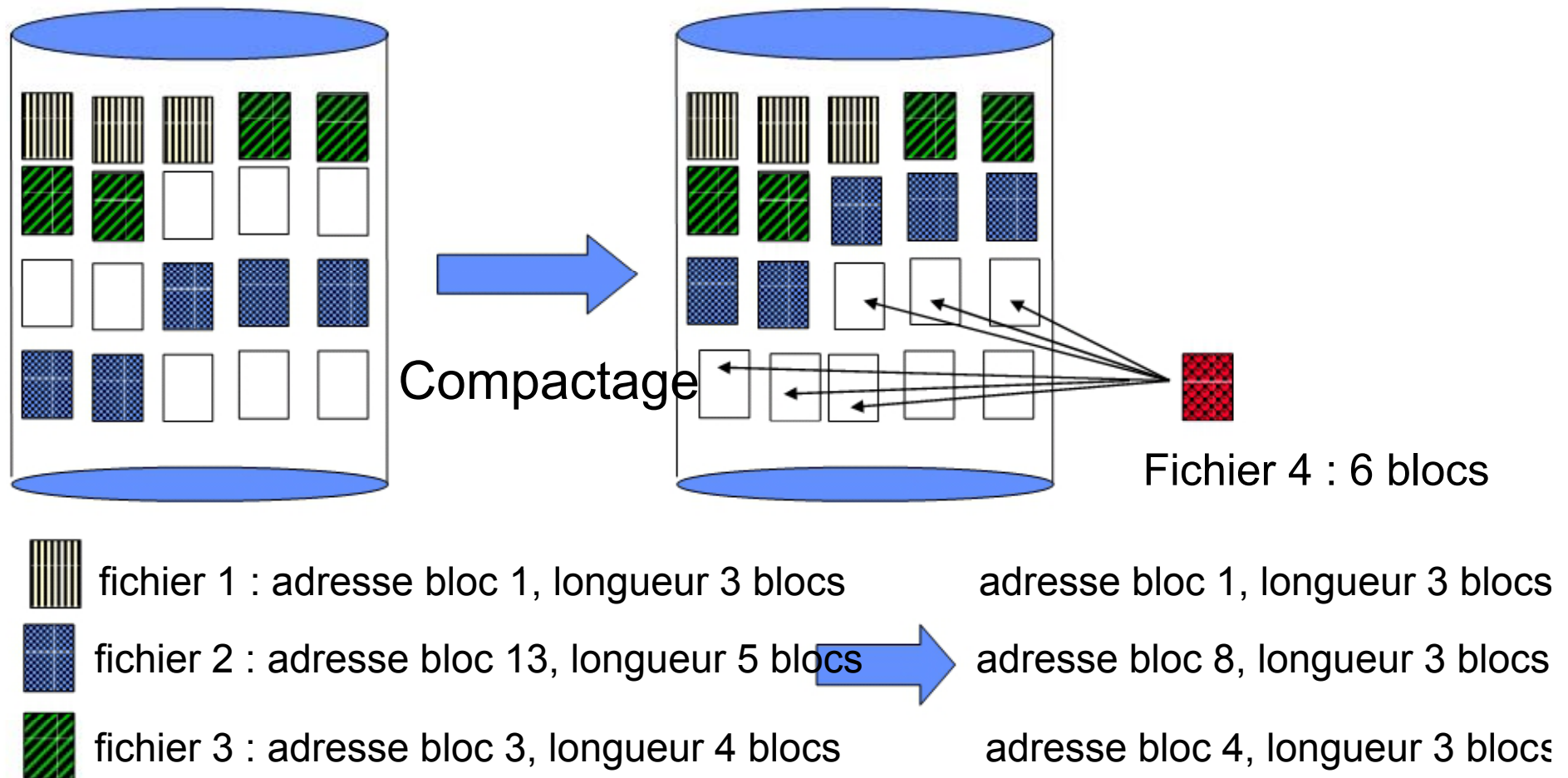


fichier 2 : adresse bloc 13, longueur 5 blocs



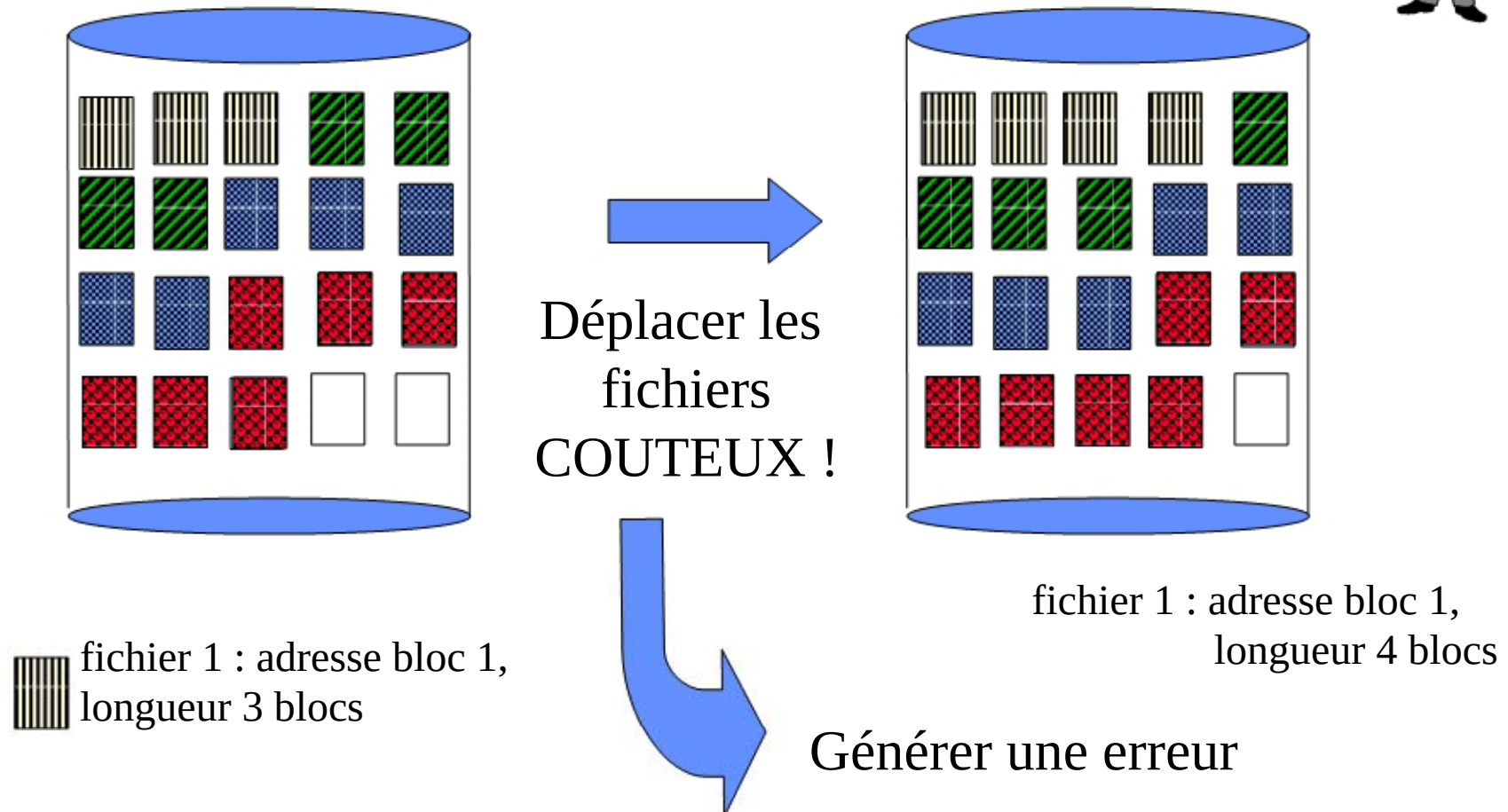
Allocation contiguë

- fragmentation de la mémoire: ramasse miette



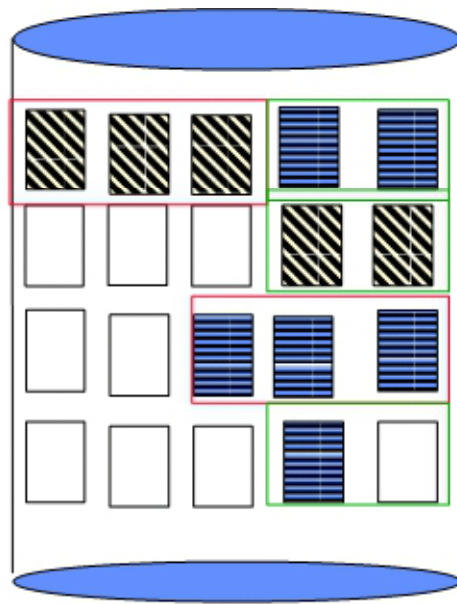
Allocation contiguë

- Etendre le fichier 1 avec un bloc de données



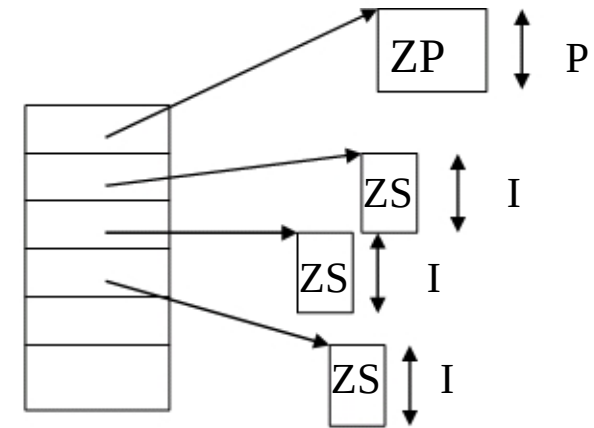
Allocation par zones

- Un fichier est constitué de plusieurs zones physiques disjointes (exemple système MVS IBM) :
 - une zone primaire allouée à la création
 - des zones secondaires (extensions) allouées au fur et à mesure des besoins
 - Chaque zone est allouée de façon indépendante



Zone
Primaire
(3 blocs)

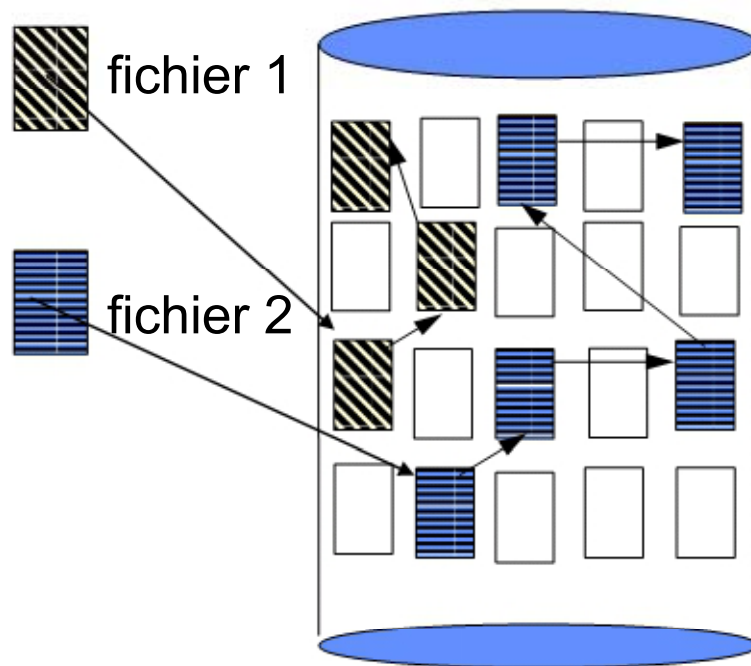
Zone
Secondaire
(2 blocs)



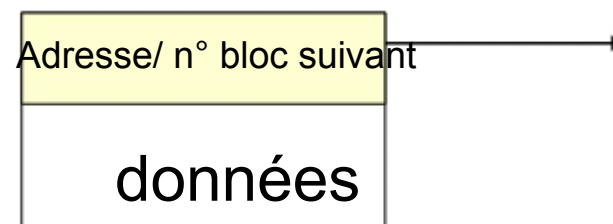
⇒ Problèmes précédents atténués
mais toujours existants

Allocation par bloc chaînée

- Un fichier est constitué comme une liste chaînée de blocs physiques, qui peuvent être dispersés n'importe où.



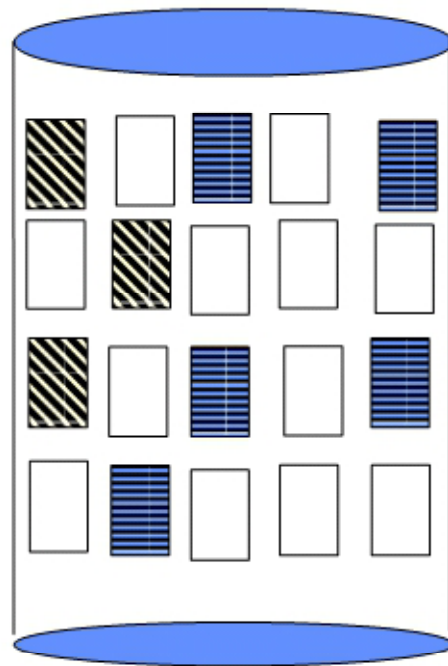
- Extension simple du fichier : allouer un nouveau bloc et le chaîner au dernier
- Pas de fragmentation
- Difficultés :
 - mode séquentiel seul
 - le chaînage du bloc suivant occupe de la place dans un bloc



Allocation par bloc chaînée : variante

- Une table d'allocation des fichiers (File allocation table - FAT) regroupe l'ensemble des chainages.

(exemple systèmes windows)



fichier 1



fichier 2

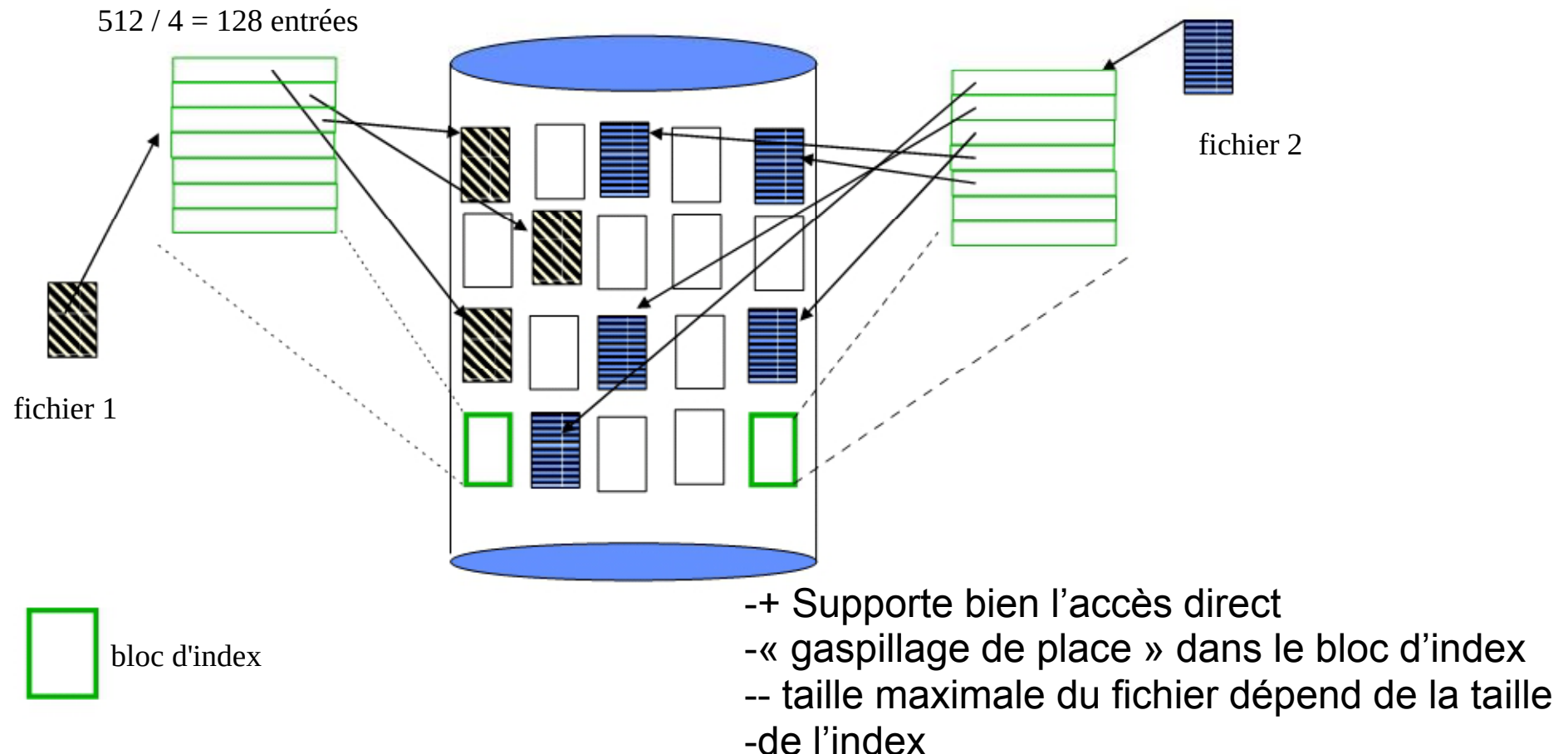


FAT

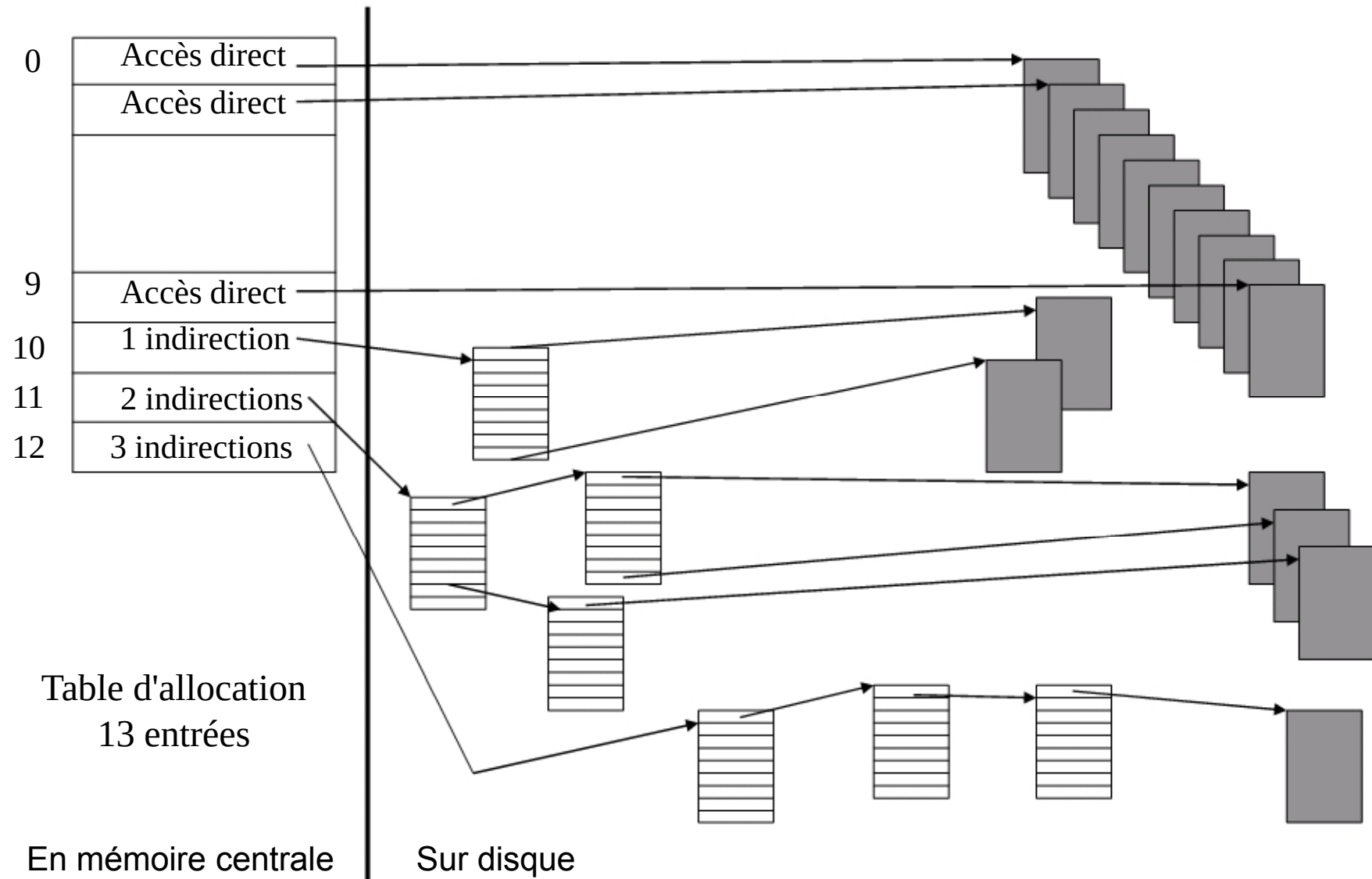
N° bloc		
1	NULL	Fin de fichier
2	Libre	Bloc non alloué
3	5	
4	Libre	
5	NULL	
6	Libre	
7	1	
		N° de Bloc suivant Alloué pour le fichier
11	7	
12	Libre	
13	15	
14	Libre	
15	3	
16	Libre	
17	13	

Allocation indexée

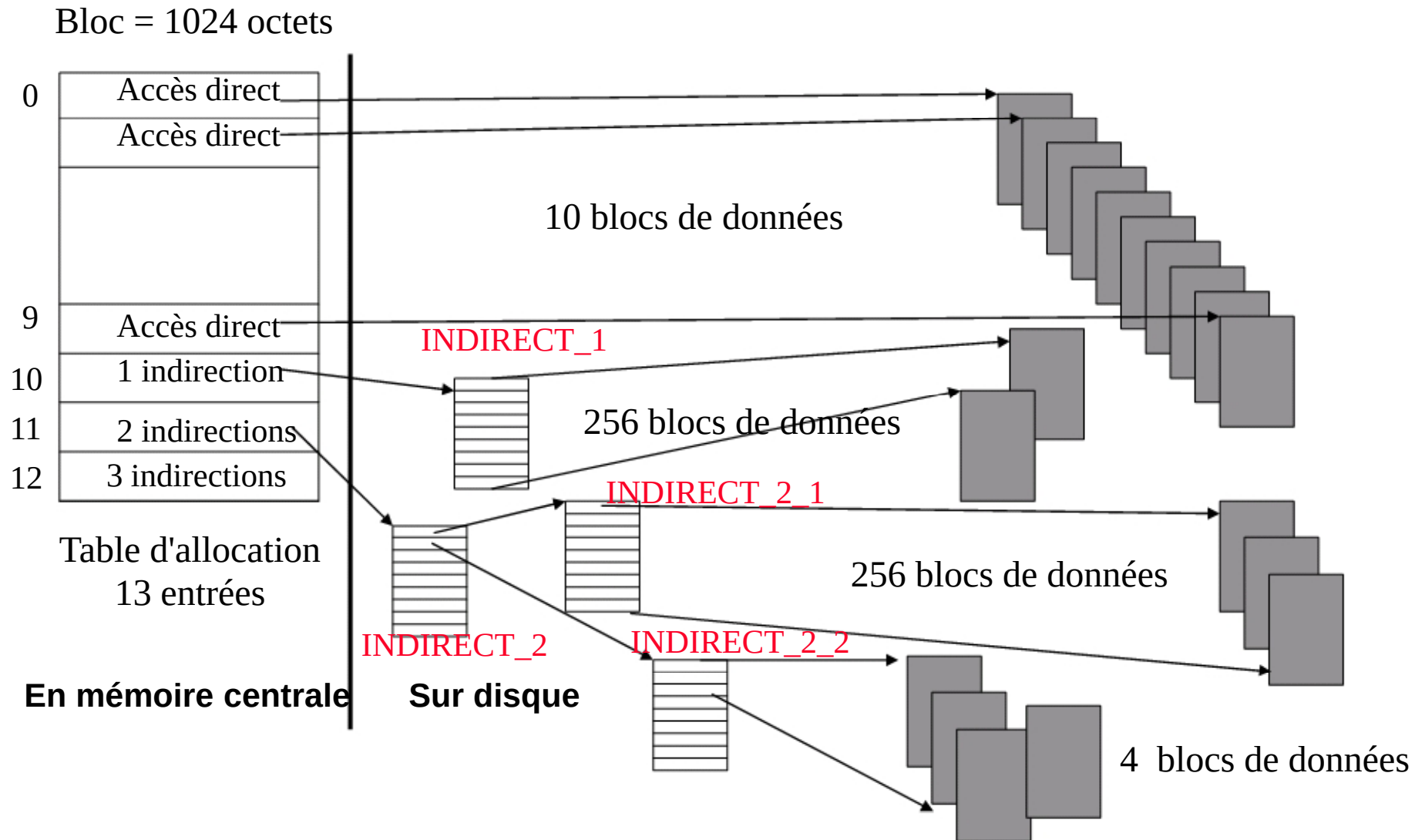
- Les adresses des blocs physiques constituant un fichier sont rangées dans une table appelée index, elle-même contenue dans un ou plusieurs blocs disque



Allocation indexée : la solution Unix/Linux



Allocation indexée : la solution Unix/Linux



Gestion de l'espace libre

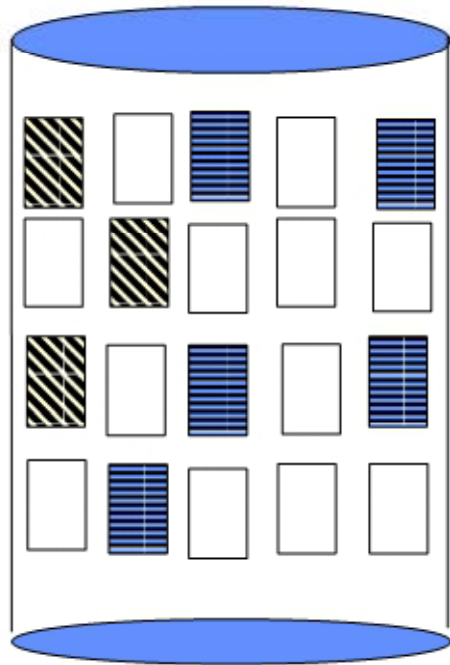
- Le système maintient une liste d'espace libre, qui mémorise tous les blocs disque libres (non alloués)
 - Création d'un fichier : recherche dans la liste d'espace libre de la quantité requise d'espace et allocation au fichier : l'espace alloué est supprimé de la liste
 - Destruction d'un fichier : l'espace libéré est intégré à la liste d'espace libre

Il existe différentes représentations possibles de l'espace libre

- vecteur de bits
- liste chaînée des blocs libres

Gestion de l'espace libre par un vecteur de bits

- La liste d'espace libre est représentée par un vecteur binaire, dans lequel chaque bloc est figuré par un bit.
 - Bloc libre : bit à 1
 - Bloc alloué : bit à 0



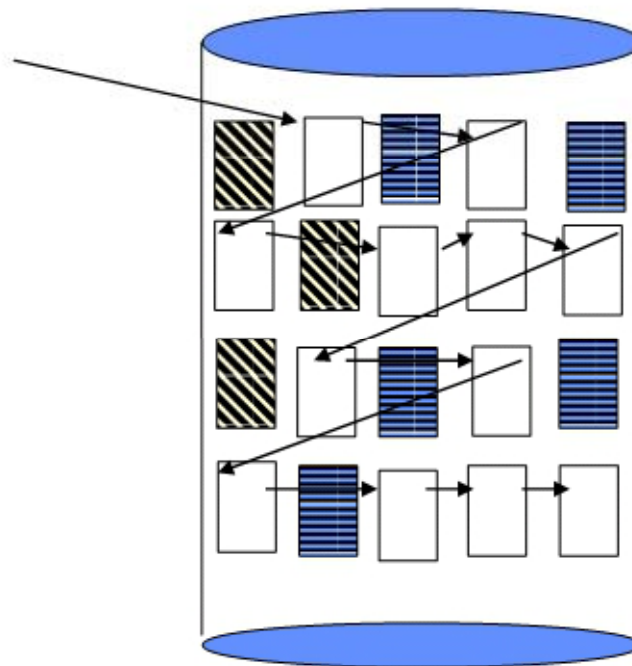
01010101110101010111

- Facilité de trouver n blocs libres consécutifs
- Système Macintosh

Gestion de l'espace libre par liste chaînée

- La liste d'espace libre est représentée par une liste chaînée des blocs libres

Liste des blocs libres

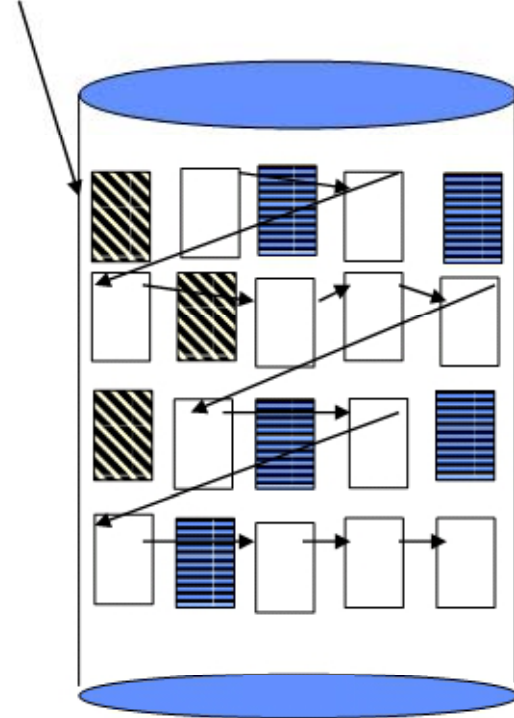


- Parcours de la liste coûteux
 - Difficile de trouver un groupe de blocs libres
- Variante par comptage

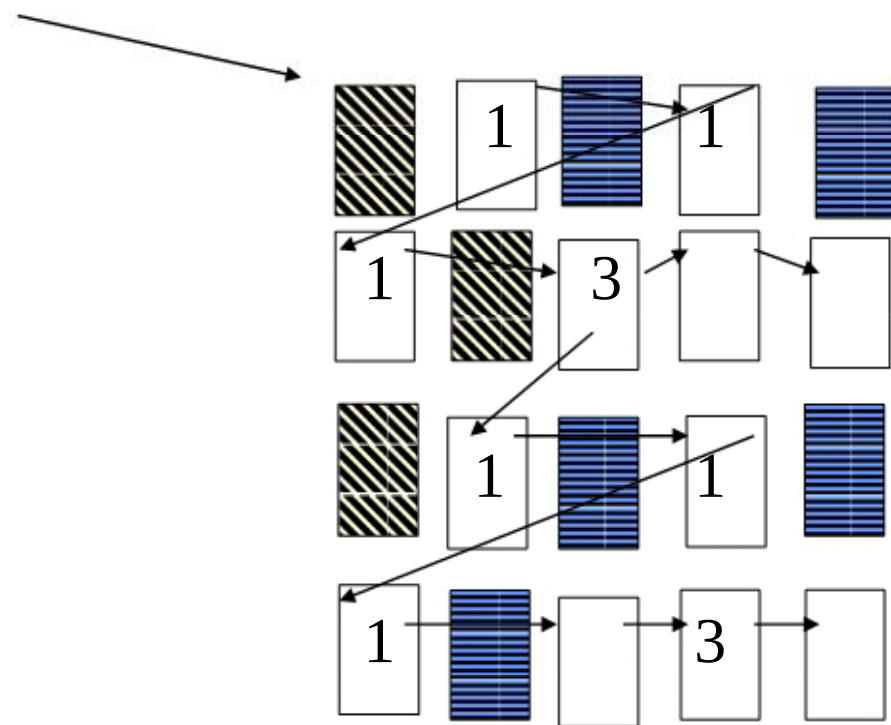
Gestion de l'espace libre par liste chaînée: variante avec comptage

- Le premier bloc libre d'une zone libre contient l'adresse du premier bloc libre dans la zone suivante et le nombre de blocs libres dans la zone courante.

Liste des blocs libres

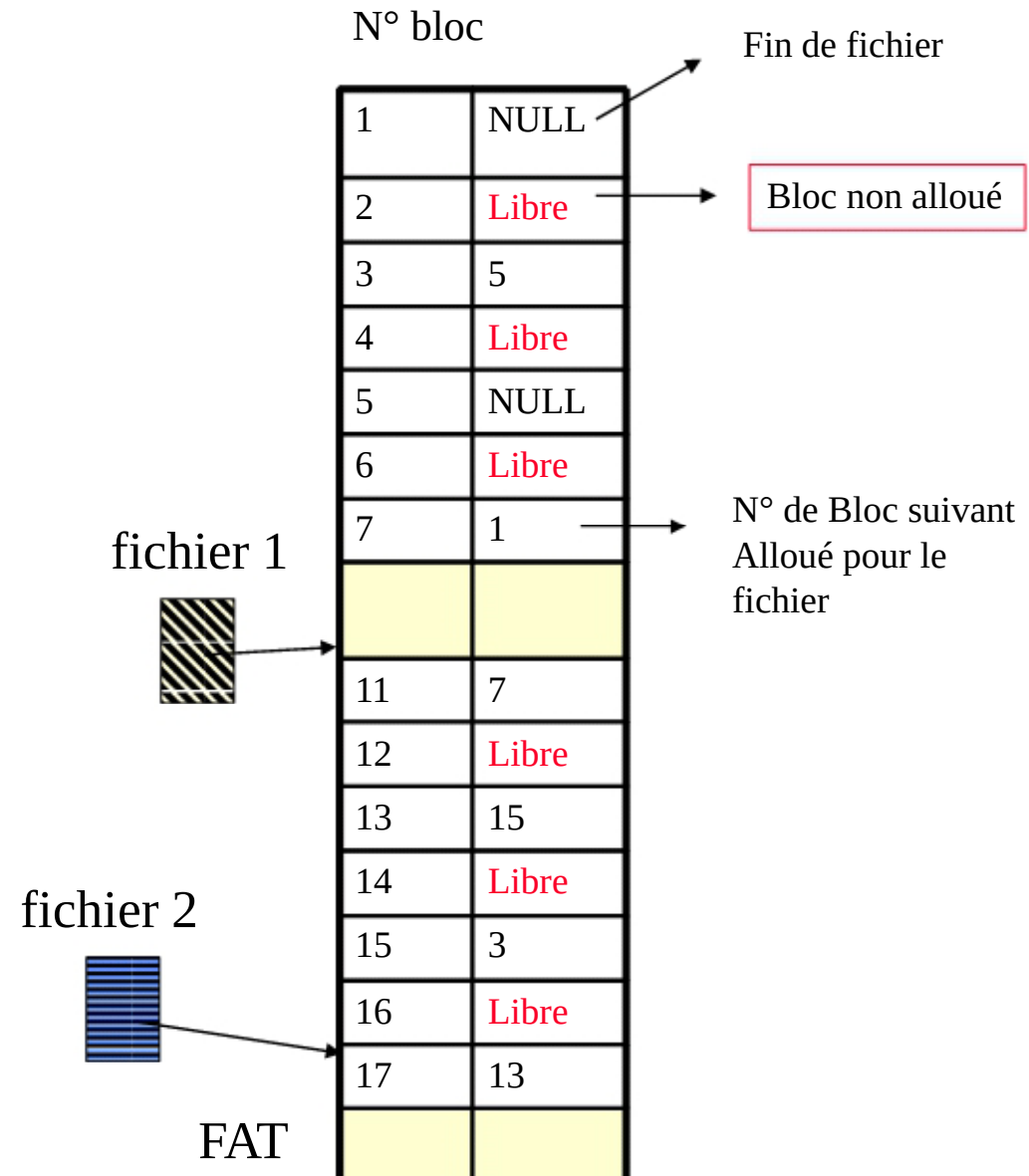
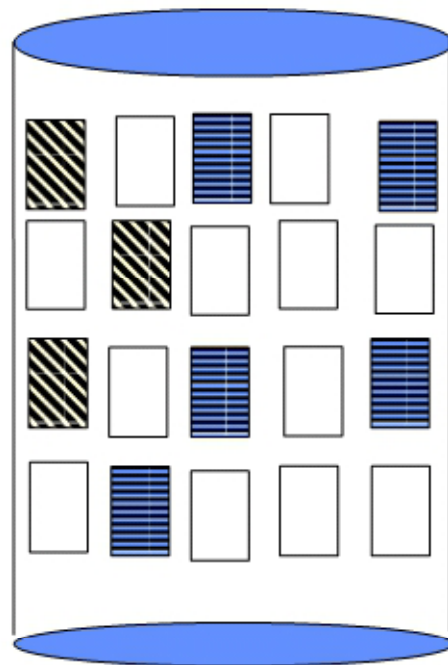


Liste des blocs libres



Gestion de l'espace libre

- La FAT intègre directement la gestion de cet espace.

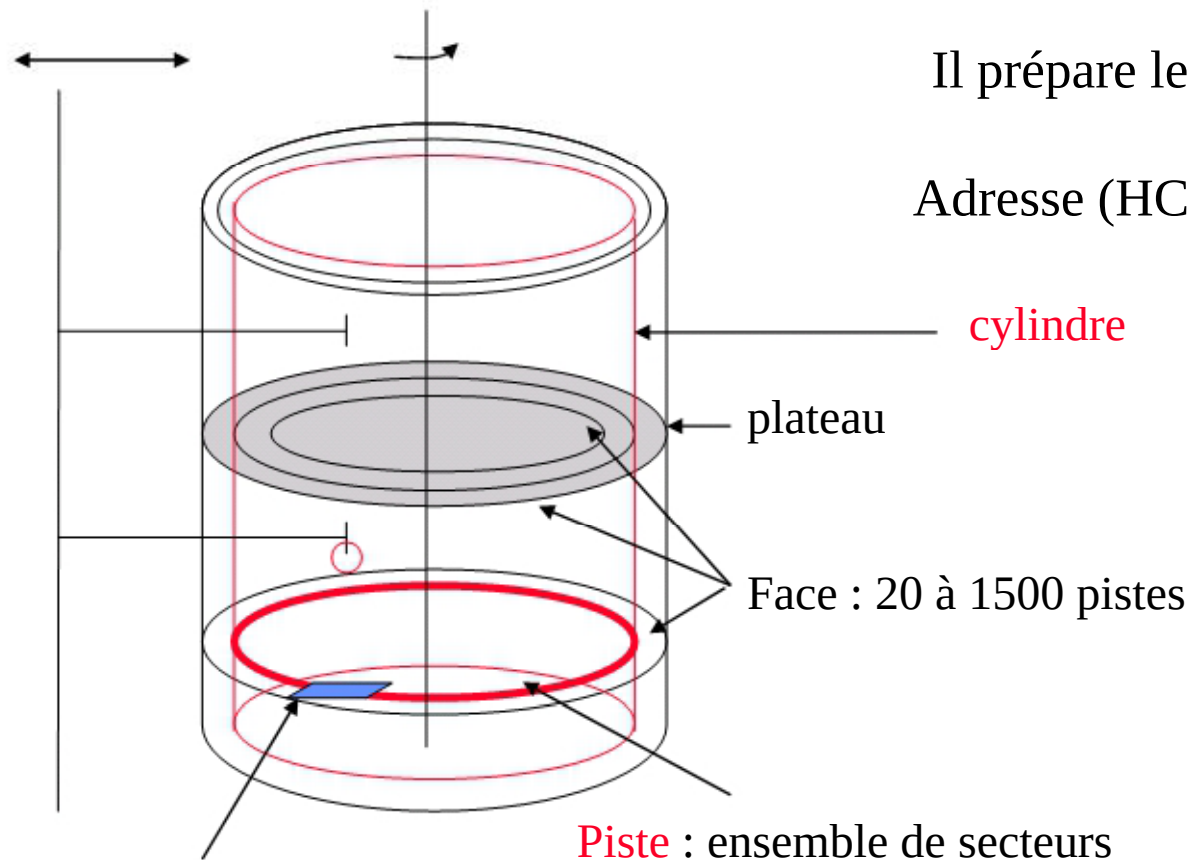


Structuration du disque dur

- Trois opérations pour structurer le disque dur :
 - Formatage physique
 - Formatage logique
 - Partitionnement

Formatage physique

- Le formatage physique ou de bas niveau permet de diviser la surface du disque en éléments basiques.

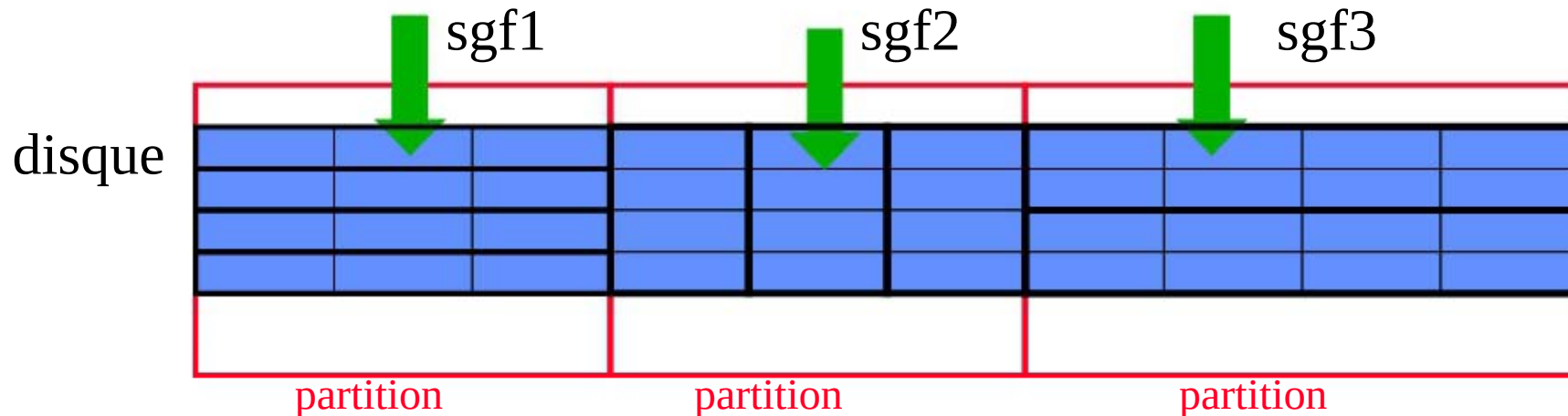


Il prépare le disque à accueillir des données
Adresse (HCS, head cylinder sector)

Secteur : 512 octets
Adresse HCS (3, 1, 30)

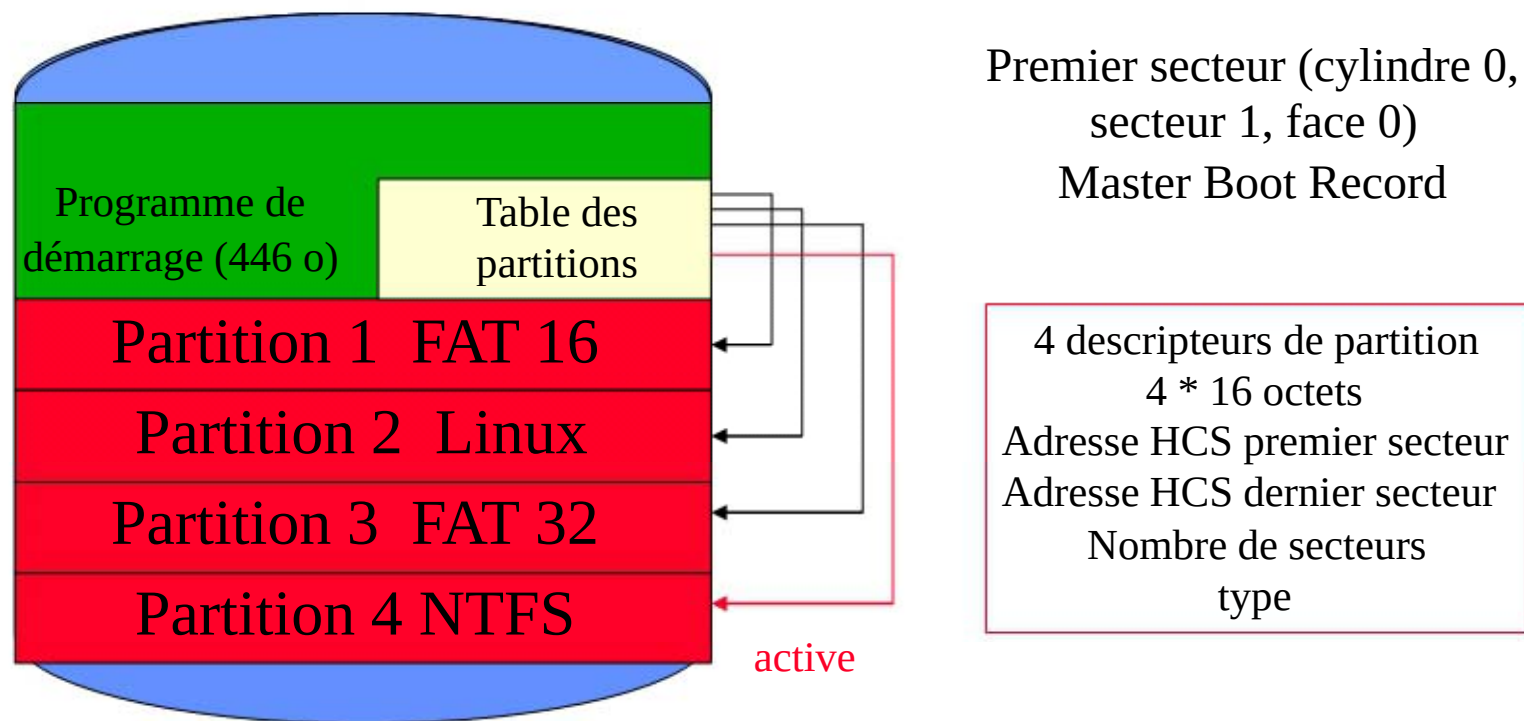
Formatage logique/ Partitionnement

- Le formatage logique ou de haut niveau crée un système de gestion de fichiers sur le disque.
 - Le type de SGF installé dépend du système d'exploitation. Il forme les clusters ou blocs
 - Il est possible d'installer plusieurs types de SGF sur un disque grâce au **partitionnement** du disque..

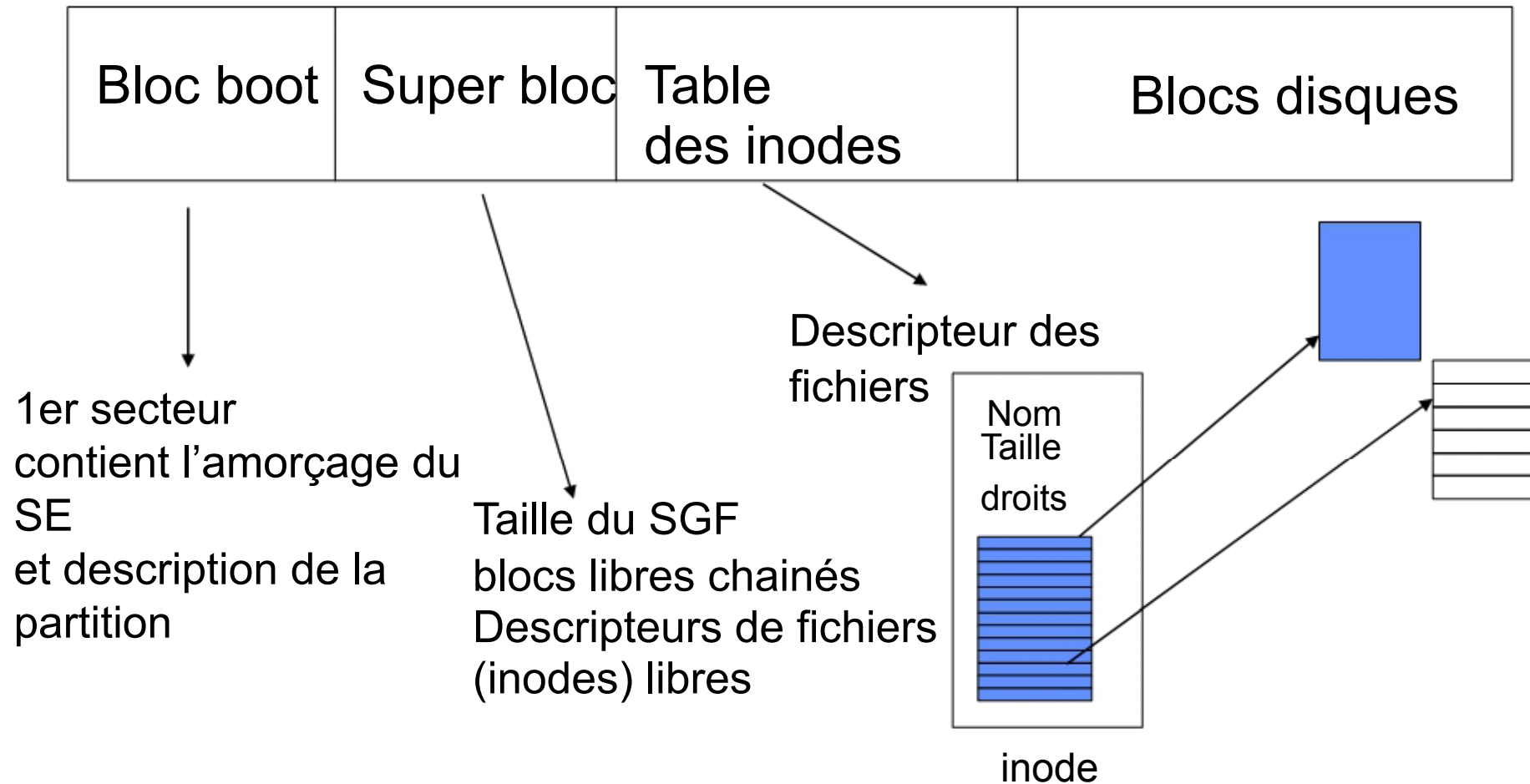


Partitionnement

- Une partition est une partie d'un disque dur destinée à accueillir un SGF. Elle est identifiée par un nom appelé « nom de volume ». Elle est constituée d'un ensemble de cylindres contigus.
- Un disque peut accueillir 4 partitions différentes. Une seule est **active** à la fois.

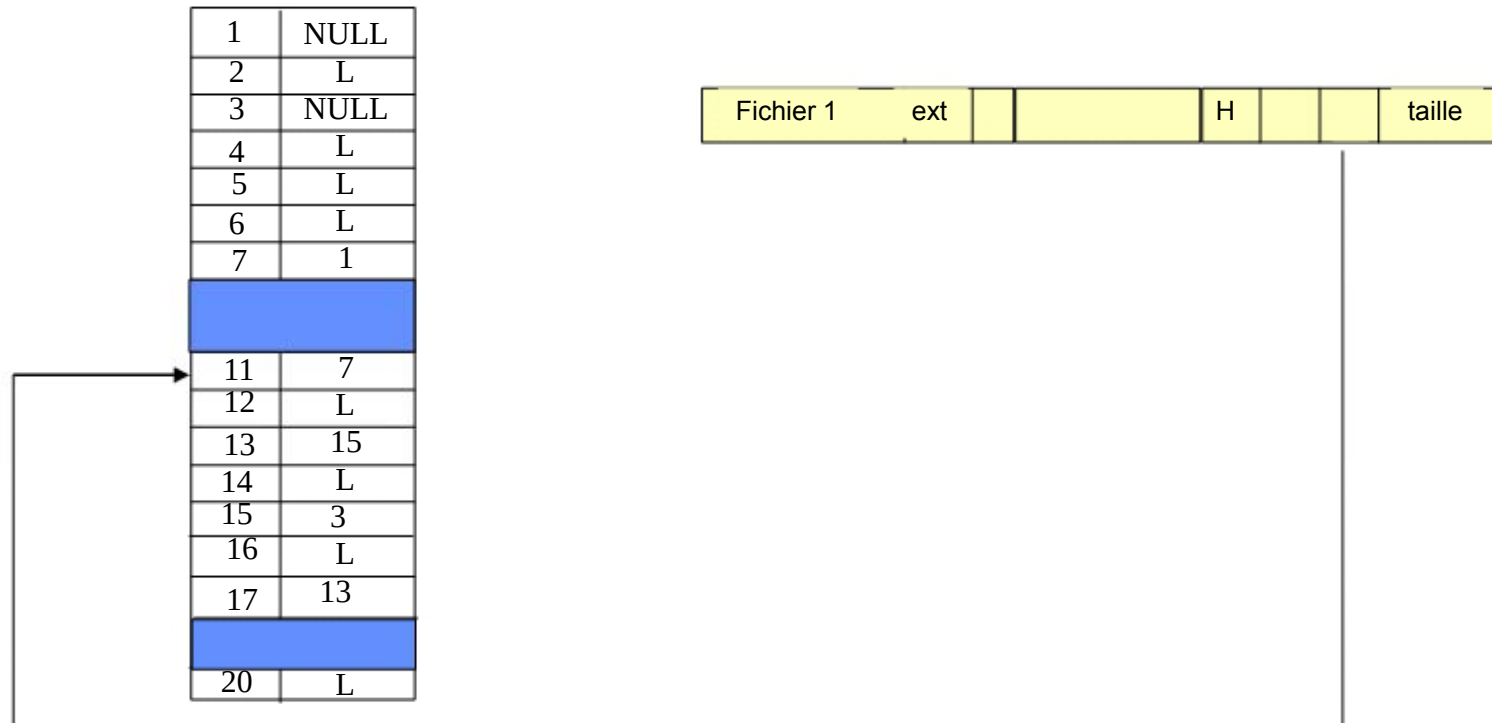


Organisation de partition : LINUX



Organisation de partition : DOS

Secteur d'amorçage	FAT	Copie de la FAT	Répertoire racine	Blocs de données
--------------------	-----	-----------------	-------------------	------------------



Partitions

Gestion de l'ordinateur

Fichier Action Affichage Fenêtre ?

← → 📁 📄 📌 📎 📏

Volume	Disposition	Type	Système de fichiers	Statut	Capacité	Espace libre	% Libres	Tolérance de pannes	Délai
(C:)	Partition	De base	NTFS	Sain (Système)	259,03 Go	240,44 Go	92 %	Non	0%
(E:)	Partition	De base	FAT	Sain (Actif)	980 Mo	383 Mo	39 %	Non	0%
	Partition	De base	NTFS	Sain	34,88 Go	23,59 Go	67 %	Non	0%
(G:)	Partition	De base	NTFS	Sain	117,79 Go	82,16 Go	69 %	Non	0%
SAUVEGARDE	Partition	De base	FAT32	Sain	39,06 Go	29,20 Go	74 %	Non	0%

Disque 0
De base
298,10 Go
Connecté

(C:) 259,03 Go NTFS Sain (Système)	SAUVEGARDE 39,07 Go FAT32 Sain
--	--------------------------------------

Disque 1
De base
152,66 Go
Connecté

(G:) 117,79 Go NTFS Sain	34,88 Go NTFS Sain
--------------------------------	-----------------------

Disque 2
Amovible
980 Mo
Connecté

(E:) 980 Mo FAT Sain (Actif)

■ Partition principale

Démarrage de l'ordinateur



1. L'utilisateur appuie sur le bouton d'alimentation de l'unité centrale

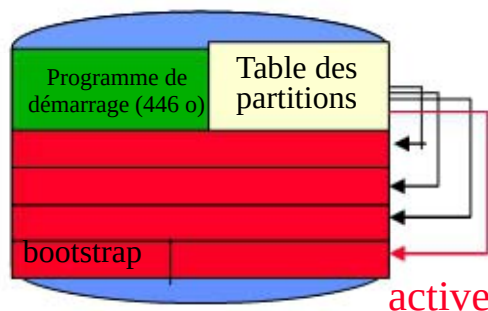


2. Une fois le courant stabilisé, le processeur démarre et exécute le code du BIOS stocké dans la ROM à une adresse prédéfinie



3. Le BIOS exécute une séquence de vérification des composants (mémoire, vidéo, périphériques de base) (POST : Power-OnSelf Test)

4. Le BIOS accède au CMOS pour lire la configuration matérielle de la machine (date, heure, **périphérique de masse contenant le système d'exploitation**).



5. Le BIOS accède au MBR du disque ; il charge en mémoire centrale le programme de démarrage

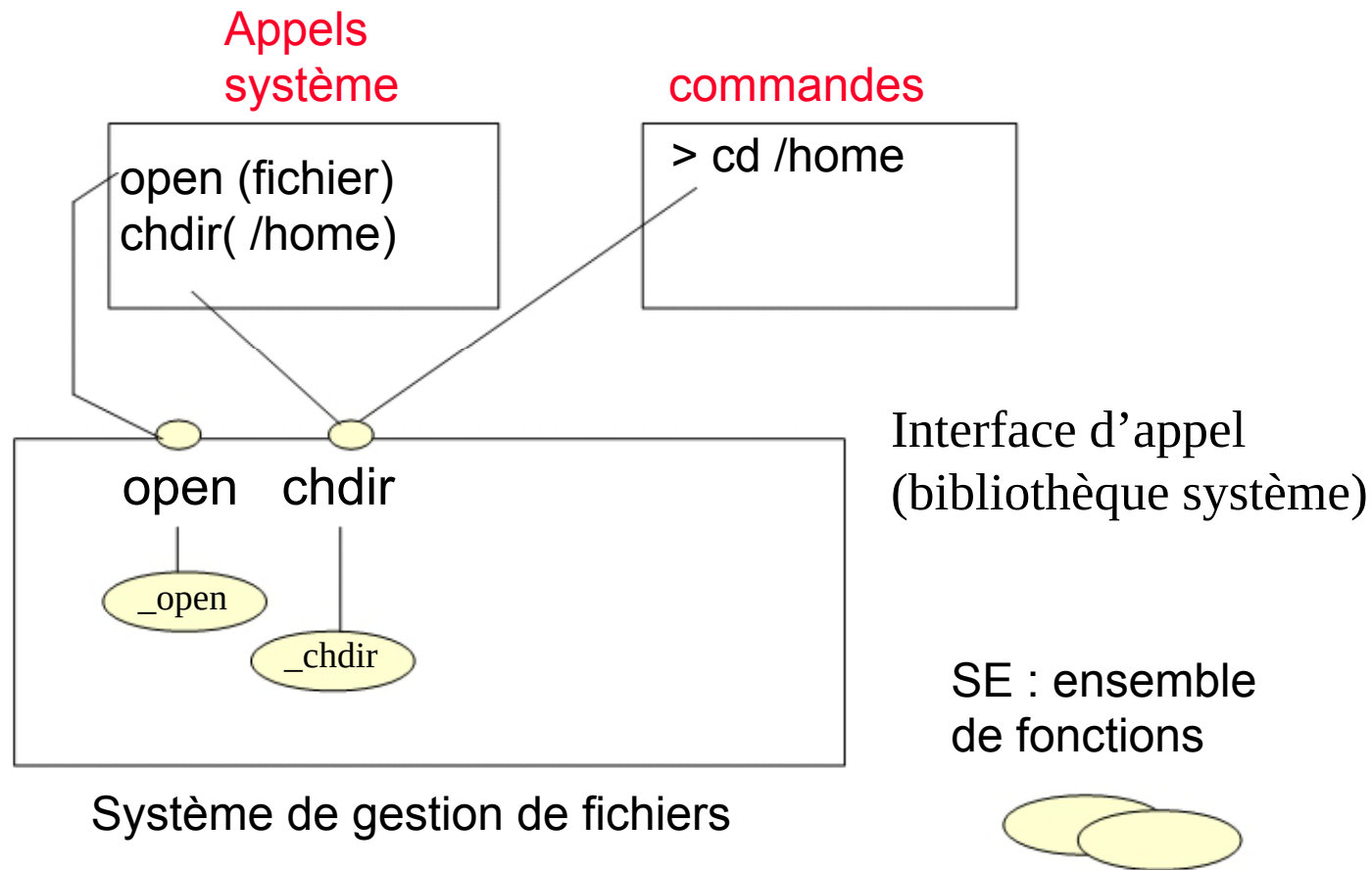
6. Le programme de démarrage détermine la partition active et transfère le contrôle au bootstrap de la partition pour charger le système d'exploitation

Montage Démonstration

Réalisation des fonctions d'accès élémentaires

Interfaces d'appel du SGF

Programme Utilisateur Interpréteur de commandes



Les commandes

Quelques commandes du SGF:

- liste du répertoire (ls, dir)
- changement de répertoire (cd)
- création répertoire (mkdir)
- suppression répertoire (rmdir)
- suppression fichier (rm, del)
- modification d'attributs d'un fichier (chmod)
- changement de nom de fichier... (mv, ren)

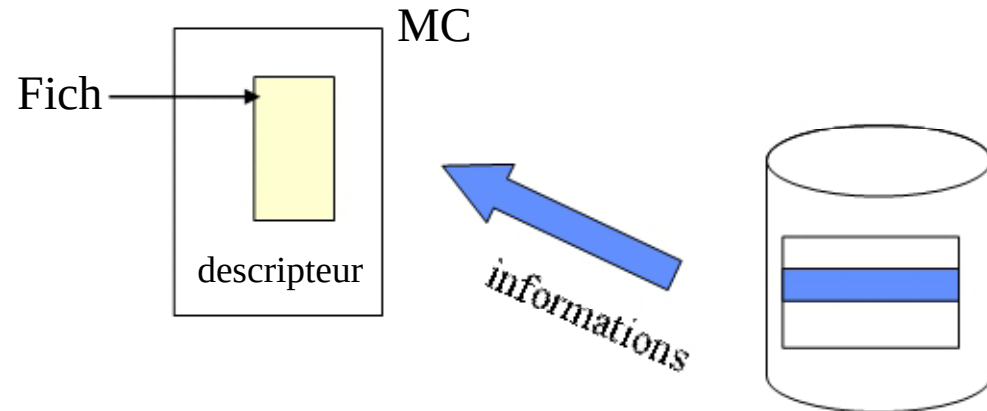
Au lancement d'une commande -> appel à la fonction du SGF

Les appels systèmes

- Quelques appels systèmes du SGF:
- ouverture fichier (open)
- création de fichier (create)
- fermeture fichier (close)
- et
- Lecture (read)
- Écriture (write)

Ouverture de fichier

Ouverture fichier



Fich = OPEN (nom_fichier, L/E)

L'ouverture

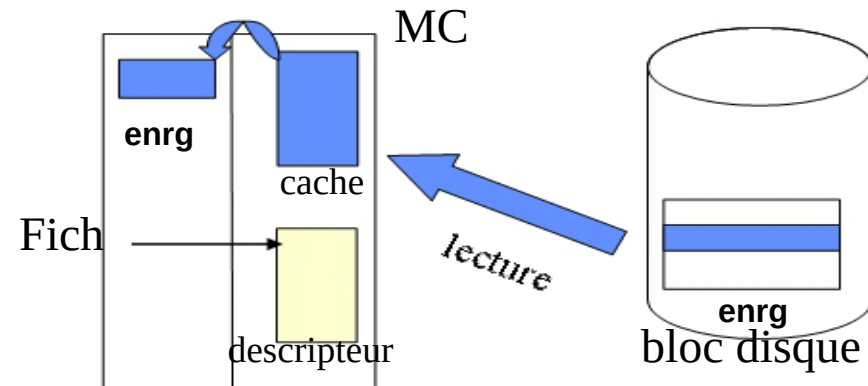
- vérifie l'existence du fichier dans le répertoire du disque
- vérifie la compatibilité des types d'accès au fichier (lecture/écriture)
- transfère les éléments de l'entrée de répertoire dans le descripteur correspondant en mémoire
- renvoie au programme un index Fich qui désigne le descripteur de fichier

Le descripteur est conservé et mis à jour en mémoire jusqu'à la fermeture

Lecture de fichier

Lecture fichier

READ (Fich, enrg, n°enregistrement)



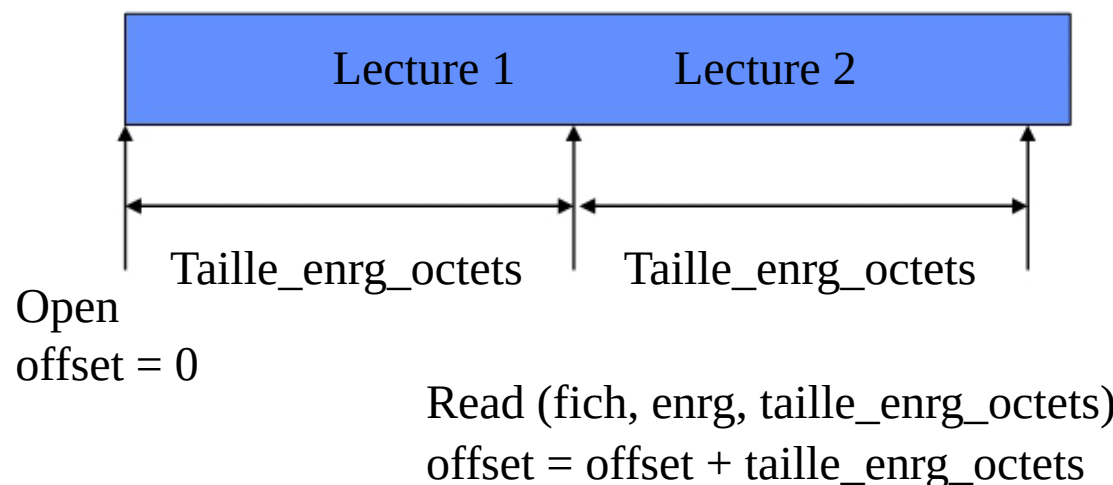
La lecture d'un fichier nécessite:

- pointeur du descripteur
- l'adresse mémoire de la zone réceptrice de l'enregistrement
- n° de l'enregistrement qui peut:
 - ne pas exister si lecture séquentielle
 - . être le rang si accès direct

Il faut déterminer l'adresse du bloc physique contenant l'enregistrement à lire
Les données lues sont transférées dans un tampon mémoire

Lecture fichier : Unix

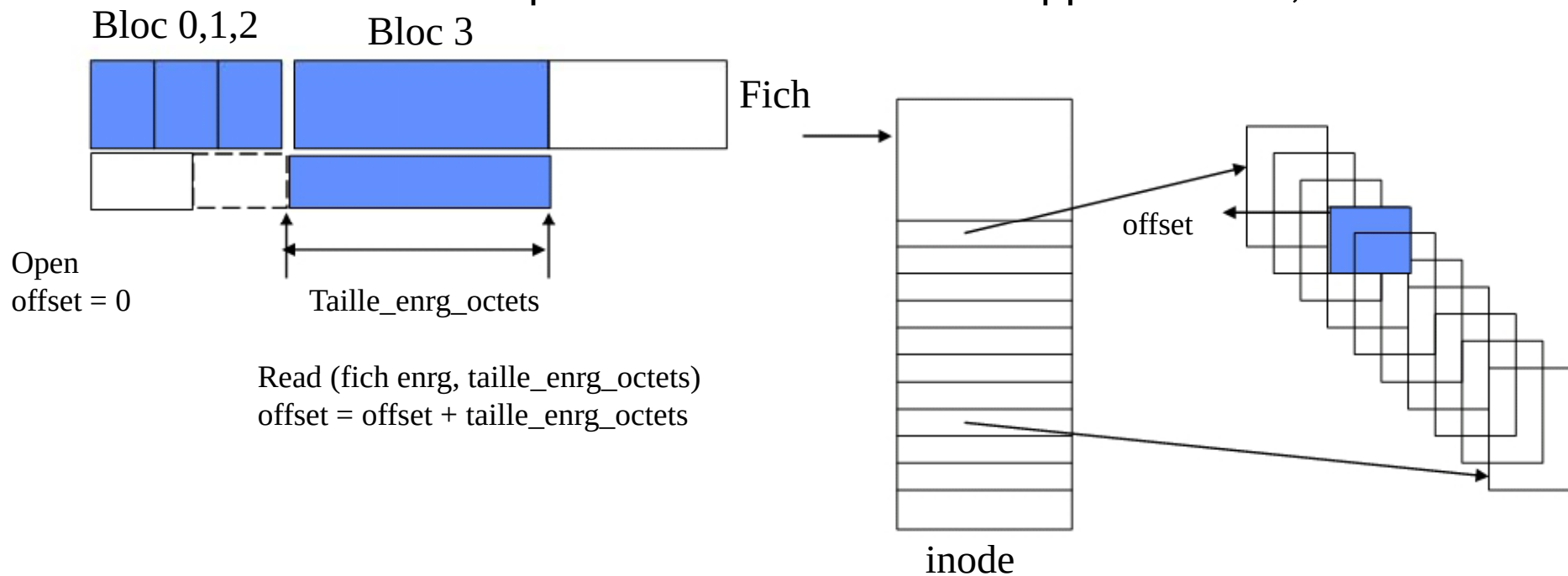
- `READ (Fich, enrg, taille_enrg_octets)`
- Un fichier Unix est une suite d'octets sans structure, accessibles de manière séquentielle.
- Le SGF maintient pour chaque fichier un offset qui pointe sur l'octet courant dans le fichier.
- Une opération de lecture spécifie combien d'octets doivent être lus (`taille_enrg_octets`).
- La lecture s'effectue depuis la position courante de l'offset et délivre les `taille_enrg_octets` octets suivants.



Lecture fichier : Unix

- Une opération de lecture spécifie combien d'octets doivent être lus (`taille_enrg_octets`).
- La lecture s'effectue depuis la position courante de l'offset et délivre les `taille_enrg_octets` octets suivants.

👉 Il faut déterminer à quel bloc les octets à lire appartiennent, et lire ce bloc



Ecriture fichier

WRITE (fich, tampon, n°enregistrement)

L'écriture dans un fichier nécessite:

- pointeur du descripteur
- l'adresse mémoire de la zone émettrice de l'enregistrement
- n° de l'enregistrement qui peut:
 - . ne pas exister si écriture séquentielle
 - . être le rang si accès direct

Cette fonction met à jour certains éléments du descripteur ; elle peut entraîner l'allocation d'un nouveau bloc au fichier.

Les données sont transférées du tampon mémoire vers le fichier

Fermeture du fichier

CLOSE (fich)

La fermeture provoque:

- transfère les éléments du descripteur vers le répertoire sur le disque
- Libération de la mémoire occupée par les tampons

Sécurité et Protection des fichiers

- Protection contre les dégâts physiques



FIABILITE



- ➔ Utilisation de différents types de redondance

- Protection contre les accès inappropriés



PROTECTION

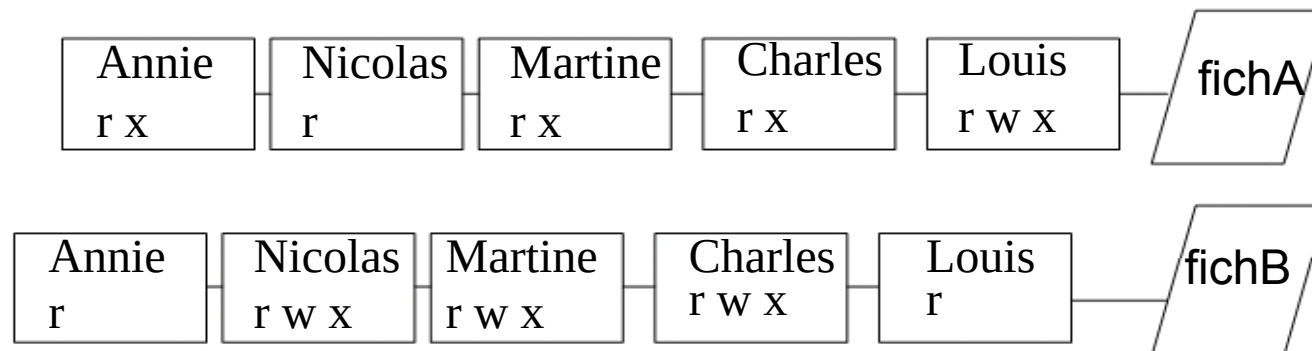


- ➔ droits d'accès

Protection contre les accès inappropriés



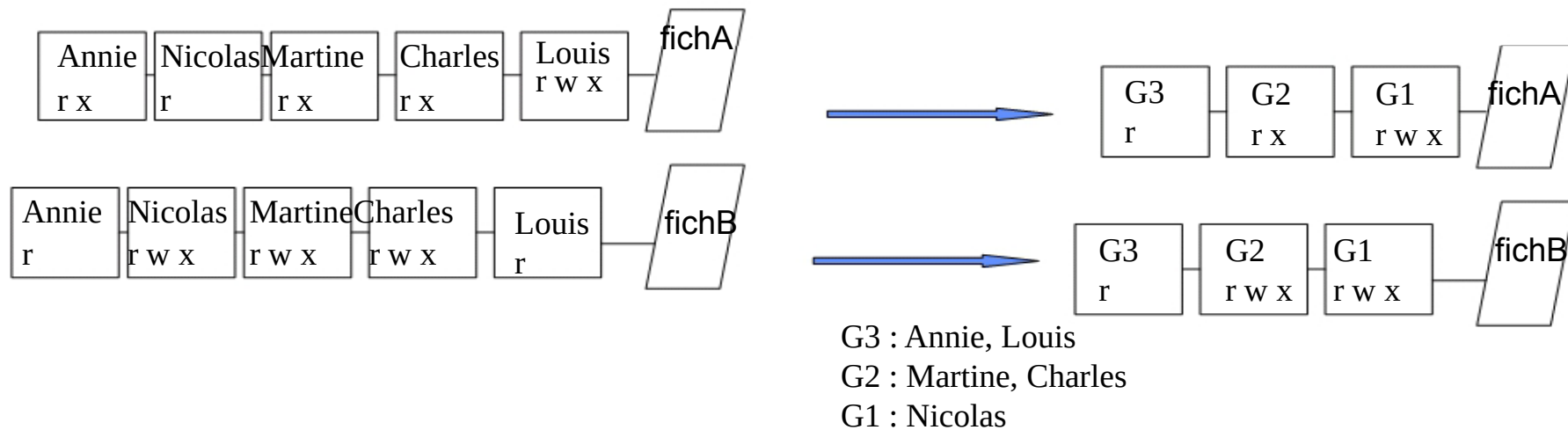
- Définition de droits d'accès
 - lecture (r), écriture (w), exécution (x), destruction ...
- A chaque fichier est associé une liste d'accès, spécifiant pour chaque utilisateur, les types d'accès qui lui sont autorisés



Protection contre les accès inappropriés



- La liste d'accès peut être longue et difficile à gérer
 - définition de groupes auxquels sont associés des droits
 - un utilisateur hérite des droits du groupe auxquels il appartient



Protection contre les accès inappropriés

- Exemple Unix

- Définition de trois groupes :

- le propriétaire : celui qui a créé le fichier
 - le groupe : le groupe de travail
 - les autres : tous les autres



> ls -l

drwxr-xr-x 2 delacroix 4096 Oct 22 1998 repertoire

-rw-r--r-- 1 delacroix 6401 Jan 8 1997 eleve.c

-rwxr-xr-x 1 delacroix 24576 Dec 15 1998 essai

-rw-r--r-- 1 delacroix 67 Dec 15 1998 essai.c

> chmod a+w essai.c

> ls -l essai.c

-rw-rw-rw- 1 delacroix 67 Dec 15 1998 essai.c

Protection contre les dégâts physiques



- Utilisation de la redondance interne :

- ☞ l'information existe en double exemplaire : une version primaire, une version secondaire
- ☞ le système maintient la cohérence entre les deux versions

☞ exemple : Windows dispose de deux exemplaires de la FAT

Secteur d'amorçage	FAT	Copie de la FAT	Répertoire racine	Blocs de données
--------------------	-----	-----------------	-------------------	------------------

primaire secondaire



Maintien de la cohérence entre les deux exemplaires

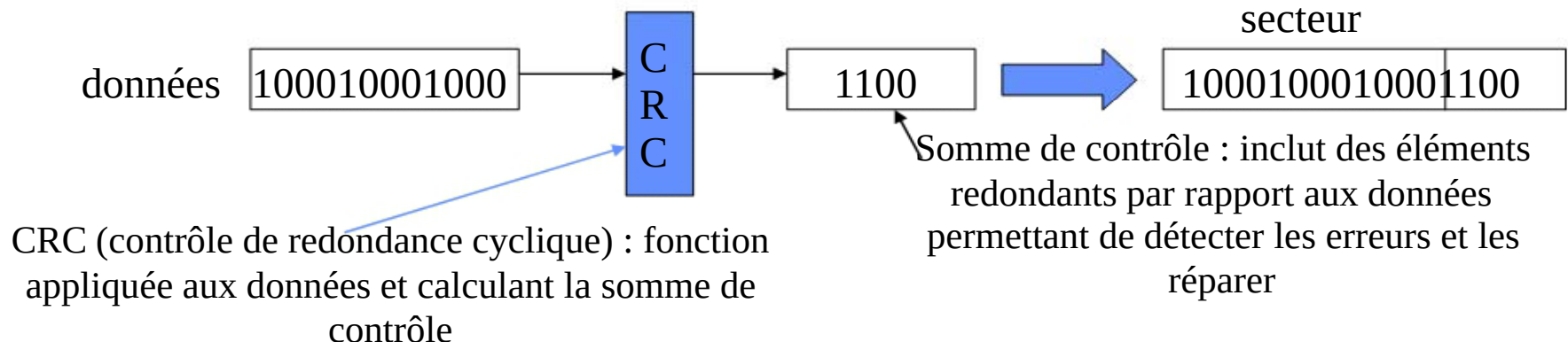
Protection contre les dégâts physiques

- Utilisation de la redondance interne :



➡ Des données supplémentaires appelées **somme de contrôle** sont ajoutées à l'information qui permettent de vérifier la validité des informations.

➡ exemple : chaque secteur du disque comporte les données + une somme de contrôle. Cette somme de contrôle permet de détecter les secteurs défectueux.



Protection contre les dégâts physiques



- Utilisation de la redondance :

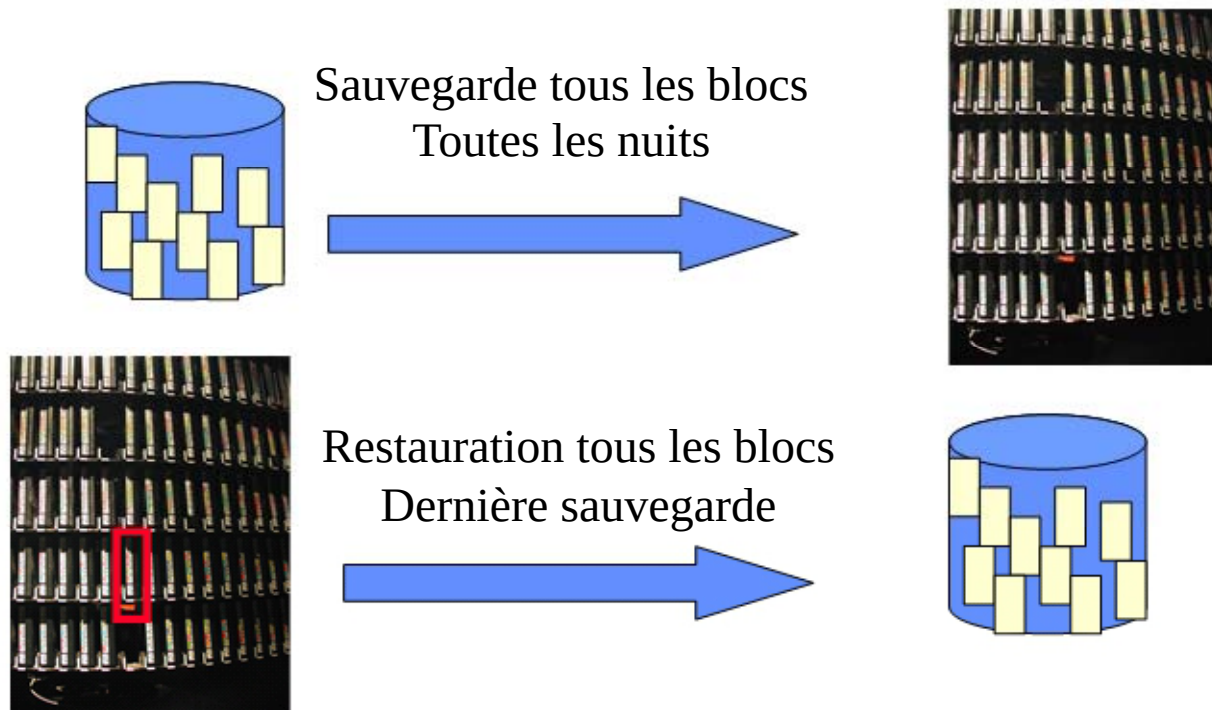
- ➡ Les données sont dupliquées sur plusieurs supports physique
- ➡ Redondance par sauvegarde périodique complète ou incrémentale
- ➡ Principes des disques RAID (Redundant Array of Independent Disk)

Protection contre les dégâts physiques



- Redondance par sauvegarde périodique :
 - ☞ sauvegarde complète : la totalité du SGF est dupliqué même si les fichiers n'ont pas été modifiés

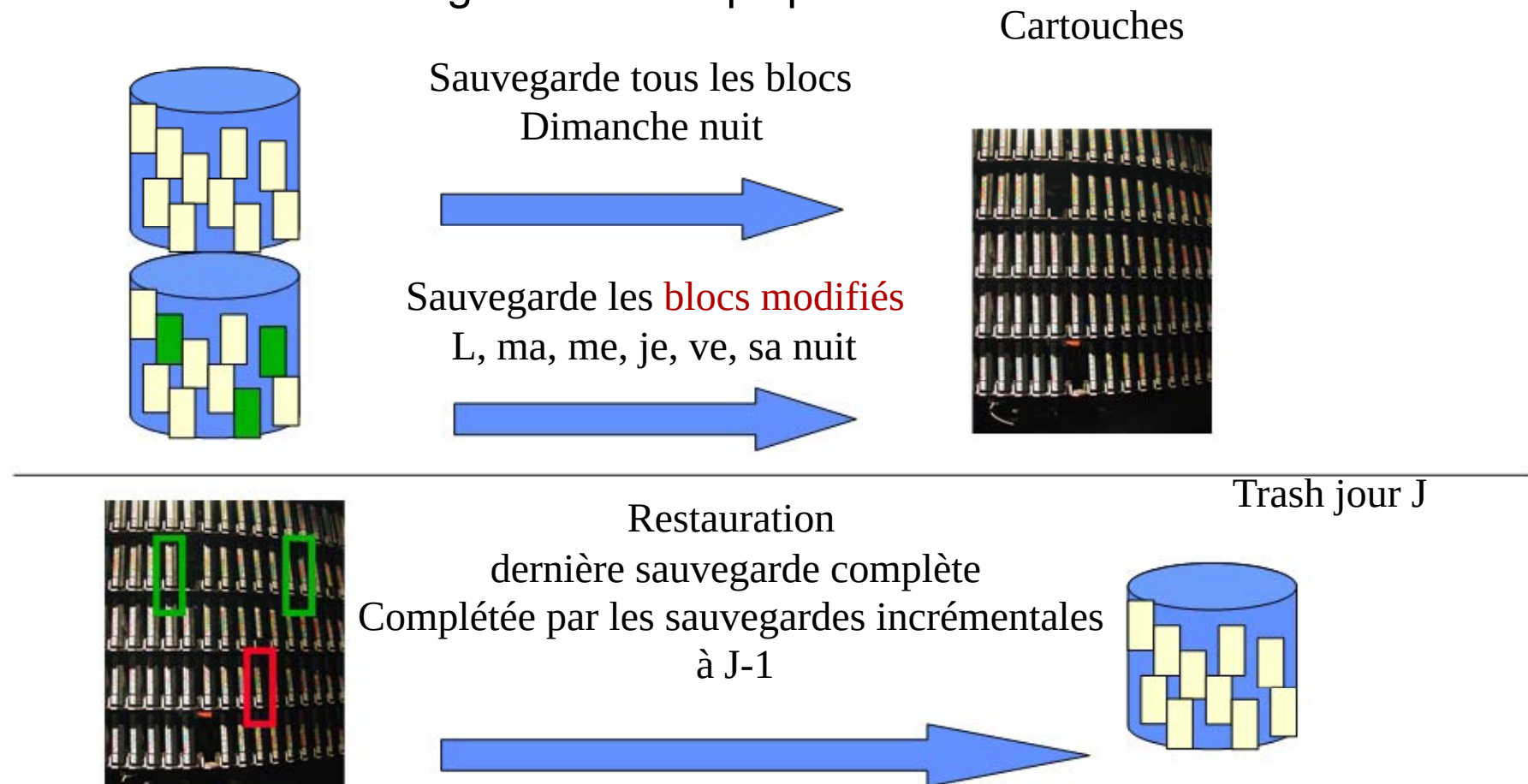
Cartouches



Protection contre les dégâts physiques



- Redondance par sauvegarde périodique :
 - ☞ sauvegarde incrémentale : seuls les objets modifiés depuis la dernière sauvegarde sont dupliqués.



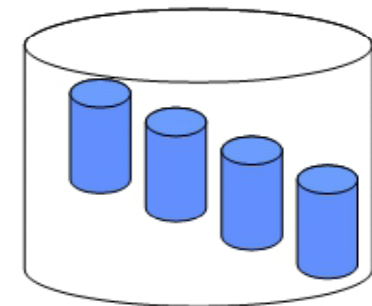
Protection contre les dégâts physiques

- Principes des disques RAID :

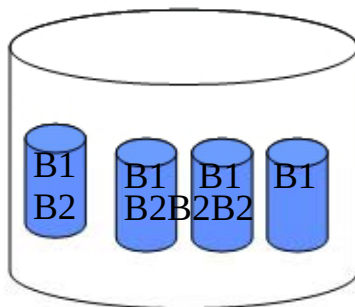
- ☞ L'unité de stockage est constituée de plusieurs disques durs constituant une grappe.



Lecture/écriture

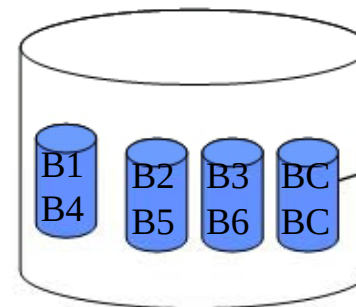


☞ RAID Niveau 1



Les données (blocs) sont dupliquées sur chaque disque

☞ RAID Niveau 4



Bloc de contrôle = $f(B1, B2, B3)$
Bloc de contrôle = $f(B4, B5, B6)$

Les données (blocs) sont répartis sur n-1 disques de la grappe
Le bloc n stocke une information redondante des données permettant de les reconstituer en cas de perte

Exemple: Le SGF d'UNIX

Rappel

- Un **fichier** est un flot d'octets non structurée, stockée sur une mémoire auxiliaire.
- Plusieurs sortes de fichiers:
 - les fichiers ordinaires;
 - les répertoires;
 - les périphériques
 - ...
- Un fichier peut avoir plusieurs liens (noms)
- Les caractéristiques statiques d'un fichier sont stockées dans un descripteur appelé *i-noeud*:
 - le type de fichier (ordinaire, répertoire, ...), taille, dates, UID-GID, liens, la liste des blocs , ...
- Un *répertoire* est un fichier particulier : c'est en fait un tableau à deux colonnes, nom, i-noeud.

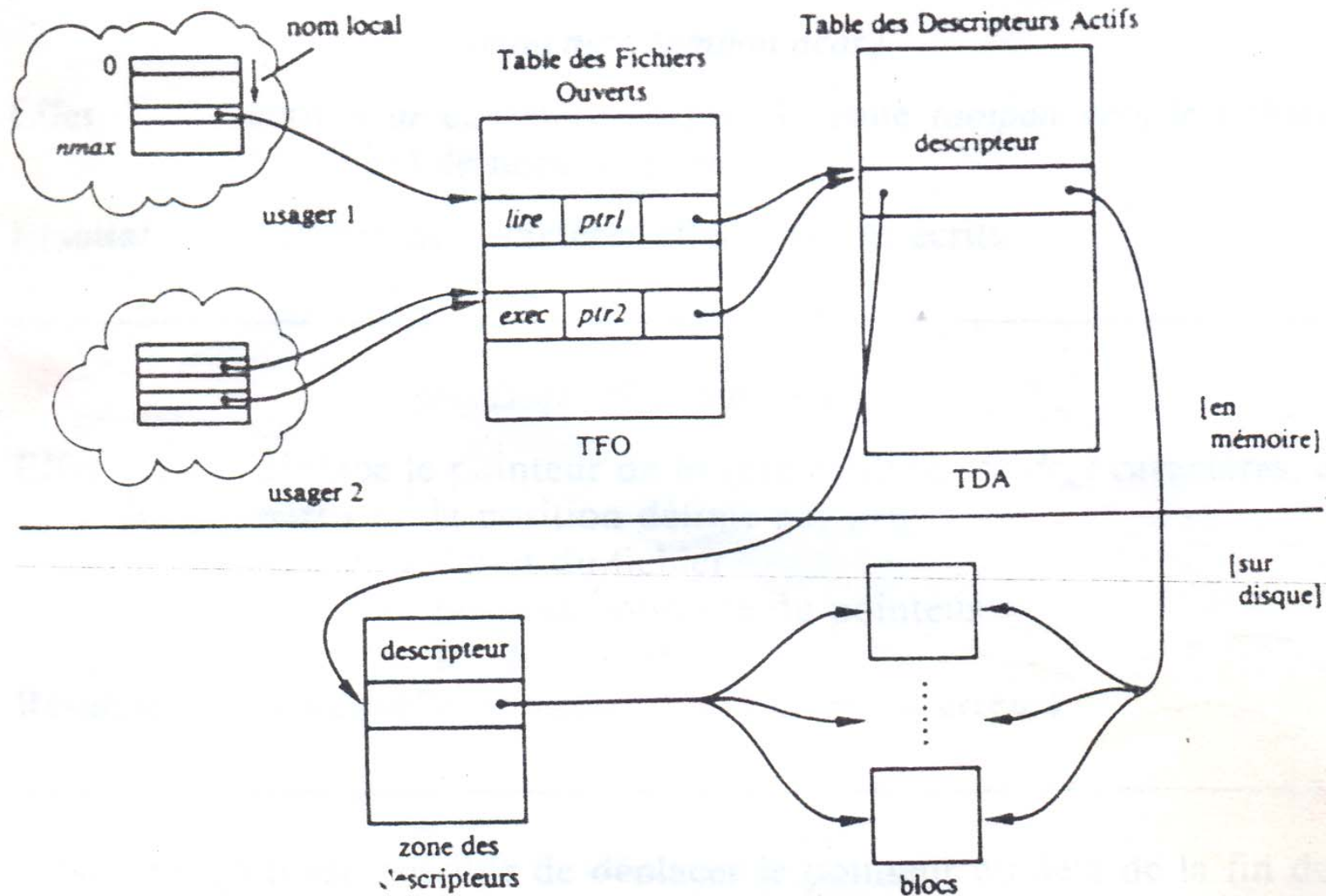
Gestion des périphériques

- Un périphérique est un fichier spécial qui possède 2 index:
 - majeur identifie les périphériques de même type
 - Mineur identifie individuellement chaque périphérique
- Un volume de stockage n'est accessible que s'il est monté: intégré le volume dans la racine du SGF (/)
- La commande **mount** permet de relier une partition ou un périphérique à un répertoire par lequel les données sont accessibles.

`mount -t iso9660 /dev/cdrom /media/cdrom`

*Le cdrom est accessible à travers le répertoire
/media/cdrom*

Fonctionnement du SGF UNIX



Les entrées sortie de base

- **Les descripteurs de fichiers**

- Toute entrée-sortie est une lecture ou une écriture d'un fichier.
- Avant d'entreprendre une opération de lecture ou d'écriture , il est nécessaire d'ouvrir le fichier (ou de le créer)
- Le système , retourne un entier non nul, appelé nom local, qui servira à le désigner.

- **Primitives E/S sur fichier -read et -write**

```
#include <unistd.h>
```

```
ssize_t read(int fd, void *buf, size_t count)  
ssize_t write(int fd, const char *buf, size_t  
count)
```

read et **write** fournissent en retour le nombre d'octets réellement transférés. Zéro indique une fin de fichier, et -1 , une erreur (*errno*).

Les entées sortie de base

- Exemple : version primitive de cat

```
/* copie son entrée sur sa sortie */  
# define SIZE 512 /* arbitraire */  
main()  
{ char  buf[SIZE];  
  int n;  
  while((n=read(0,buf,sizeof(buf)) > 0)  
        write(1, buf, n);  
  exit(0);  
}
```

Les entées sortie de base

- **Ouverture de fichier**

`int open(const char *pathname, int flags)`

- **open** retourne un nom local de fichier (entier) , -1 si une erreur.
- **mode** d'ouverture sont définis dans **/usr/include/fcntl.h**:
 - O_RDONLY** ouverture en lecture seulement;
 - O_WRONLY** ouverture en écriture seulement;
 - O_RDWR** ouverture en lecture et écriture;
- D'autres bits peuvent être combinés (ET) avec les bits précédents:
 - O_NDELAY** ouverture non bloquante (tubes ,fichiers spéciaux);
 - O_APPEND** écriture uniquement en fin de fichier
 - O_CREAT** création de fichier s'il n'existe pas(fournir **droits**)
 - O_TRUNC** effacer le contenu du fichier s'il existe.
 - O_EXCL** avec **O_CREAT**, provoque un echec si il existe
- Une autre forme:

`int open(const char *pathname, int flags, mode_t droits)`

droits droits d'accès (création seulement).

Cette valeur est modifiée par le **umask** du processus : la véritable valeur utilisée est **(mode & ~umask)**.

Les entées sortie de base

- **La création de fichier**

`int creat(const char *pathname, mode_t droits)`

creat, utilisée pour créer de nouveau fichier ou pour en réécrire d'anciens.

- retourne un nom local utilisé lors des opérations futures sur le fichier; -1 en cas d'erreur
- **creat** ouvre le fichier en écriture .

- **Fermeture de fichiers.**

`int close(int fd)`

- **close** rompt le lien entre un fichier et un nom local, rendant celui-ci libre
- A sortie du programme (**exit**) ferme tous les fichiers ouverts.

- **Suppression de fichiers**

`int unlink(const char *pathname)`

unlink détruit un fichier dans le système de fichier.

Les entées sortie de base

- L'accès aléatoire –**lseek**

off_t lseek(int fd, off_t déplacement, int origine)

- **lseek** permet de se déplacer dans un fichier sans lecture/écriture à la position *déplacement* dans le fichier associé au descripteur *fd* en suivant la directive *origine* ainsi :

SEEK_SET La tête est placée à *offset octets* depuis le début du fichier.

SEEK_CUR La tête de lecture/écriture est avancée de *déplacement* octets.

SEEK_END La tête est placée à la fin du fichier plus *déplacement* octets.

- La position courante devient *déplacement*, considéré par rapport à l'origine.

```
lseek(fd, 0L, SEEK_END);  
lseek(fd, 0L, SEEK_SET);  
pos = lseek(fd, 0L, SEEK_CUR);
```

pour ajouter à la fin de fichier
pour retourner au début du fichier
position courante

Les entées sortie de base

- Duplication d'un descripteur de fichier

```
int dup(int oldfd);
```

```
int dup2(int oldfd, int newfd)
```

- **dup** et **dup2** créent une copie du descripteur de fichier *oldfd*.
- **dup** utilise le plus petit numéro inutilisé pour le nouveau descripteur.
- **dup2** transforme *newfd* en une copie de *oldfd*, fermant auparavant *newfd* si besoin est.

```
d=creat("toto", 0666);  
close((2);  
dup(d);close(d);
```

redirige la sortie standard
d'erreur sur le fichier toto.

Les répertoires

- Un répertoire décrit l'association entre le nom de fichier et le descripteur (i-nœud) .

nom de fichier	n° d'inode
----------------	------------

- Une entrée du répertoire est décrit par une structure

struct dirent

```
{  
    long d_ino;           /* inode number */  
    off_t d_off;          /* offset to this dirent */  
    unsigned short d_reclen; /* length of this d_name */  
    char d_name [NAME_MAX+1]; /* file name */  
}
```

- Accès au répertoire**

struct dirent *readdir (DIR *dir)

readdir() renvoie l'entrée suivante du flux répertoire, NULL a la fin du répertoire

DIR *opendir (const char *name);

opendir() ouvre un flux répertoire correspondant au répertoire *name*, et renvoie un *pointeur* sur ce flux. Le flux est positionné sur la première entrée du répertoire

Exemple : listdir.c Affiche le Contenu

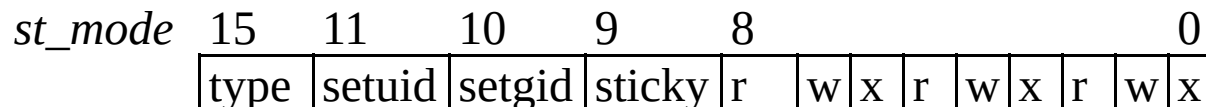
```
#include <assert.h>
#include <dirent.h>
#include <stdio.h>
#include <string.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <unistd.h>
int main (int argc, char* argv[])
{
    char* dir_path;
    DIR* dir;
    struct dirent* entry;
    char entry_path[PATH_MAX + 1];
    size_t path_len;
    if (argc >= 2) /* répertoire spécifié sur la ligne de commande s'il y a lieu. */
        dir_path = argv[1];
    else /* Sinon, utilise le répertoire courant. */
        dir_path = ".";
    strncpy (entry_path, dir_path, sizeof (entry_path)); /* Copie le chemin du répertoire dans entry_path. */
    path_len = strlen (dir_path);
    if (entry_path[path_len - 1] != '/') { /* Si le répertoire ne se termine pas par un slash, l'ajoute. */
        entry_path[path_len] = '/';
        entry_path[path_len + 1] = '\0';
        ++path_len;
    }
    dir = opendir (dir_path); /* Démarre l'affichage du contenu du répertoire. */
    while ((entry = readdir (dir)) != NULL) { /* Boucle sur les entrées du répertoire. */
        const char* type; /* chemin complet=répertoire+nom de l'entrée. */
        strncpy(entry_path+path_len,entry->d_name,sizeof(entry_path)-path_len);
        printf ("%s\n",entry_path);
    }
    closedir (dir);
    return 0;
}
```

les inodes

- Le descripteur de fichier, ou inode est décrit dans une structure **stat** (`#include <sys/stat.h>`)

struct stat

```
{  dev_t  st_dev;           périphérique du inode */
   ino_t  st_ino;          numéro du inode */
   ushort st_mode          type du fichier, attributs, résumé
   short  st_nlink;        nombre de liens
   ushort st_uid;          numéro du propriétaire */
   ushort st_gid;          numéro du groupe */
   dev_t  st_rdev;         n° de ressource pour fichier spécial */
   off_t  st_size;         taille du fichier en octets */
   time_t st_atime;        date de dernier accès
   time_t st_mtime;        date de dernière modification
   time_t st_ctime;        date de dernière modif. de caract
};
```



les inodes

- Accès au descripteur

```
int stat(const char *file_name, struct  
stat *buf);
```

récupère le statut du fichier pointé par *file_name* et remplit le buffer *buf*

```
int fstat(int fildes, struct stat *buf)
```

identique à **stat**, sauf que le fichier ouvert est pointé par le descripteur *fildes*

```
int lstat(const char *file_name, struct  
stat *buf);
```

identique à **stat**, sauf qu'il donne le statut d'un lien lui-même et non pas du fichier pointé par ce lien